



DEPARTMENT OF
COMPUTER SCIENCE

JOÃO EDUARDO GOMES PIRES GARÇÃO
BSc in Computer Science and Engineering

AN INTEGRATED FRAMEWORK FOR THE DEVELOPMENT OF CRDTS

MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN DISTRIBUTED AND PARALLEL SYSTEMS

NOVA University Lisbon

Draft: September 22, 2026

AN INTEGRATED FRAMEWORK FOR THE DEVELOPMENT OF CRDTS

JOÃO EDUARDO GOMES PIRES GARÇÃO

BSc in Computer Science and Engineering

Supervisor: Prof. Dr. Mário José Parreira Pereira

Assistant Professor, NOVA University Lisbon

Co-supervisor: Prof. Dr. Carla Maria Gonçalves Ferreira

Full Professor, NOVA University Lisbon

MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN DISTRIBUTED AND PARALLEL SYSTEMS

NOVA University Lisbon

Draft: September 22, 2026

ABSTRACT

Conflict-Free Replicated Data Types (CRDTs) are a key abstraction for modern distributed systems, enabling concurrent updates across multiple replicas while guaranteeing Strong Eventual Consistency without centralized coordination. Despite these guarantees, designing CRDTs correctly remains challenging, and formal verification plays an important role in preventing errors that compromise data integrity. Automated verification tools based on SMT solvers, such as VeriFx, enable rapid prototyping and the validation of algebraic convergence properties, but face significant limitations when reasoning about recursive definitions, inductive properties, and global state invariants. Deductive platforms such as Why3 overcome these limitations through explicit contracts and invariants, at the cost of a much higher specification effort from the programmer.

This dissertation addresses this trade-off with an integrated framework for the modular, formal development of CRDTs. Instead of translating directly between VeriFx and Why3, the framework introduces a dedicated language for specifying state-based and operation-based CRDTs. Once validated through lexical, syntactic, and type analysis over a common abstract syntax tree, a single specification is automatically compiled into both VeriFx, targeting automated proof of convergence properties via the Z3 solver, and WhyML, targeting deductive verification of semantic correctness and global invariants. This design hides the syntactic and modelling complexity of both target ecosystems from the programmer, while ensuring that the two verification efforts remain consistent with one another, since both are derived from the same specification. The framework was evaluated on ten CRDTs, including counters, sets, a generic map, and the Replicated Growable Array, consistently reducing the manual specification effort required while producing idiomatic, tool-specific artifacts for both targets. In particular, the Replicated Growable Array case study exposes a scenario in which the deductive Why3 proof succeeds while the fully automated VeriFx/Z3 proof fails, illustrating the benefit of pairing automated and deductive verification rather than relying on either in isolation.

Keywords: CRDTs · Formal Verification · Why3 · VeriFx

RESUMO

Os Conflict-Free Replicated Data Types (CRDTs) representam um conceito fundamental para sistemas distribuídos modernos, permitindo atualizações concorrentes em múltiplas réplicas ao mesmo tempo que garantem Strong Eventual Consistency sem coordenação centralizada. Apesar destas garantias, desenhar CRDTs corretamente continua a ser um desafio, e a verificação formal desempenha um papel importante na prevenção de erros que comprometam a integridade dos dados. As ferramentas de verificação automática baseadas em solvers SMT, como o VeriF_x, permitem a rápida validação experimental de propriedades algébricas de convergência, mas enfrentam limitações significativas ao raciocinar sobre definições recursivas, propriedades indutivas e invariantes de estado globais. Plataformas dedutivas como o Why3 ultrapassam estas limitações através de contratos e invariantes explícitos, ao custo de um esforço de especificação muito superior por parte do programador.

Esta dissertação responde a este compromisso com uma framework integrada para o desenvolvimento modular e formal de CRDTs. Ao invés de traduzir diretamente entre o VeriF_x e o Why3, a framework introduz uma linguagem própria para especificar CRDTs baseados em estado e em operações. Depois de validada através de análise léxica, sintática e de tipos sobre uma árvore de sintaxe abstrata comum, uma única especificação é automaticamente compilada tanto para VeriF_x, visando a prova automática de propriedades de convergência através do solver Z3, como para WhyML, visando a verificação dedutiva da correção semântica e dos invariantes globais. Este desenho oculta do programador a complexidade sintática e de modelação de ambos os ecossistemas alvo, assegurando que os dois esforços de verificação permanecem consistentes entre si, já que ambos derivam da mesma especificação. A framework foi avaliada em dez CRDTs, incluindo contadores, conjuntos, um mapa genérico e o Replicated Growable Array, reduzindo o esforço de especificação manual exigido, ao mesmo tempo que gera implementações que respeitam as convenções específicas de cada ferramenta. Em particular, o caso de estudo do Replicated Growable Array expõe um cenário em que a prova dedutiva no Why3 é bem-sucedida enquanto a prova totalmente automática no VeriF_x falha, ilustrando a vantagem de combinar verificação automática e dedutiva em vez de depender de qualquer uma isoladamente.

Palavras-chave: CRDTs · Verificação Formal · Why3 · VeriFx

DISCLOSURE OF DELEGATION TO GENERATIVE ARTIFICIAL INTELLIGENCE

I declare that Generative Artificial Intelligence (GenAI) tools were not used for research, drafting, or writing in the preparation of this work. All content is solely the result of human authorship.

CONTENTS

Abstract	ii
Resumo	iii
Disclosure of Delegation to Generative Artificial Intelligence	v
Contents	vi
List of Figures	ix
List of Tables	x
Glossary	xii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Definition	1
1.3 Goals and Expected Contributions	2
1.4 Report Structure	3
2 Related Work	4
2.1 Consistency models and Replication	4
2.1.1 Weak Consistency	5
2.1.2 Strong Consistency	6
2.1.3 Strong Eventual Consistency	7
2.2 Conflict Resolution Strategies	7
2.2.1 Ad-hoc Strategies	8
2.2.2 Conflict-free Replicated Data Types	8
2.2.3 Hybrid and Tunable Policies	9
2.3 Synchronization Models of CRDTs	9
2.3.1 Convergent Replicated Data Types	9
2.3.2 Commutative Replicated Data Types	10
2.3.3 Composition and Nesting	11
2.4 Formal Verification	11
2.4.1 Manual Verification	12

2.4.2	Automated Verification	12
2.4.3	Deductive Verification	12
2.4.4	Invariants and Security	13
2.5	Verification Tools	13
2.5.1	VeriFx	13
2.5.2	Why3	14
3	Background	15
3.1	Formalization of CRDTs	15
3.1.1	State Based Convergence	15
3.1.2	Operation Based Convergence	16
3.1.3	Pure Operation-Based CRDTs	16
3.1.4	Causality and Logical Clocks	17
3.2	Modeling and Verification in VeriFx	17
3.2.1	RDT Modeling	17
3.2.2	Automated Verification	18
3.3	Deduction and Specification in Why3	19
3.3.1	WhyML	19
3.3.2	Ghost Code and Invariants	19
3.3.3	Deductive Proofs	20
4	Language to Specify CRDTs	21
4.1	Syntax	21
4.2	The CvRDT and CmRDT Interfaces	23
4.2.1	Interface Declarations	24
4.2.2	Axioms	25
4.3	Meeting an Interface's Requirements	26
4.3.1	Validating Modules	27
4.3.2	State Representation and the payload/t Distinction	27
4.4	Backend-Specific Directives	28
4.4.1	VeriFx Functions	28
4.4.2	Shaping the VeriFx Representation	28
4.4.3	VeriFx Preconditions	29
4.4.4	Proving Termination with <code>variant</code>	30
4.5	Type System	32
4.5.1	Types	32
4.5.2	Typing Expressions	33
4.5.3	Resolving Types for Collection Calls	34
5	Compiling to VeriFx and Why3	35
5.1	Why3 Backend	35
5.1.1	Auxiliary and Main Modules	35

5.1.2	Axioms as Proof Obligations	37
5.1.3	Printing Types and Expressions	38
5.2	VeriF _x Backend	38
5.2.1	Classes and the Proof Trait	38
5.2.2	Axioms as Proof Templates	40
5.2.3	Translating Quantifiers	41
5.3	Compiling Different CRDTs	42
5.3.1	Generation Rules	42
5.3.2	Sets and Generic Maps	43
5.3.3	Composing CRDTs	43
5.3.4	Vector and Explicit State	44
5.4	Compiling RGA	44
6	Case Studies and Evaluation	46
6.1	Specification Effort	46
6.2	RGA Proof	48
7	Conclusion and Future Work	50
7.1	Conclusion	50
7.2	Future Work	51
	Bibliography	52
	Appendix A: Grow-only Counter	55
	Appendix B: Positive-Negative Counter	56
	Appendix C: Two-Phase Set	57
	Appendix D: Why3 Implementation	58
	Appendix E: CRDT Specifications	61

LIST OF FIGURES

LIST OF TABLES

6.1	Manual specification effort versus code generated, by CRDT.	47
-----	---	----

LIST OF LISTINGS

4.1	GCounter_CRDT module	23
4.2	The CvRDT and CmRDT interfaces	24
4.3	find_insert_position, as generated for VeriFx	32
5.1	GCAuxiliary and GCounter_CRDT, as generated for Why3	35
5.2	The built-in axioms, declared inside CvRDT and CmRDT	37
5.3	merge_values and its axioms, generated inside NMAuxiliary	38
5.4	GCounter_CRDT, as generated for VeriFx	38
5.5	OpCounter_CRDT, showing the execute to prepare/effect split	40
5.6	CvRDTProof, generated once and shared by every CvRDT	40
6.1	An inductive proof of find_insert_position, by fuel	49
E.1	GCounter_CRDT	61
E.2	PNCounter_CRDT	61
E.3	VGCounter	62
E.4	OpCounter_CRDT	63
E.5	NestedMap	64
E.6	GSet	65
E.7	TwoPSet	65
E.8	OpTwoPSet	66
E.9	UWMap	67
E.10	RGA	68

GLOSSARY

- Alt-Ergo** A SMT solver used as one of Why3's external provers.
- VeriFx** A language and tool for the automated, SMT-based verification of replicated data types, generating convergence proofs, verified automatically by the Z3 solver.
- Why3** A platform for deductive program verification, generating proof obligations from WhyML specifications and handing them to a choice of external provers.
- WhyML** The programming and specification language used by Why3, combining executable code with contracts, invariants, and ghost code.
- Z3** A SMT solver used as VeriFx's sole solver and as one of Why3's external provers.

INTRODUCTION

1.1 Motivation

Modern computing is inherently dependent on large-scale distributed systems, where high availability and low latency are essential requirements. To meet these demands in the presence of network failures and temporary disconnections, the CAP theorem [8] implies that strong consistency must often be relaxed in favor of availability. In this context, Conflict-Free Replicated Data Types (CRDTs) have emerged as a fundamental solution, providing Strong Eventual Consistency by construction [23]. CRDTs allow distributed replicas to be updated independently and concurrently, while guaranteeing that all replicas eventually converge to the same state without requiring synchronous coordination.

Despite these guarantees, implementing CRDTs correctly is challenging and prone to errors that can compromise data integrity. Consequently, formal verification techniques play an important role in increasing confidence in their correctness. Existing approaches can be divided into fully-automated verification and deductive verification. Automated tools based on SMT solvers, such as VeriF_x [19], support validation of convergence properties and the automatic discovery of counterexamples. Deductive verification platforms, such as Why3 [7], offer stronger guarantees by enabling proofs of functional correctness through contracts and invariants.

Experience shows that neither approach is sufficient when used in isolation. Automated verification, while efficient, struggles to handle complex or recursive structures. Deductive verification, although more expressive, often requires manual effort. This highlights the need for a methodology that combines the strengths of both approaches to support the verification of modern CRDTs.

1.2 Problem Definition

Despite advances in automated verification, tools such as VeriF_x face clear limitations when applied to CRDTs with more complex behavior. The main problem addressed in this dissertation is that purely automated techniques are often unable to guarantee

correctness in scenarios that require inductive reasoning or the preservation of global state invariants [22].

Previous work has shown that SMT-based solvers exhibit well-known limitations when reasoning about recursive definitions [7]. First, SMT-based solvers have difficulty reasoning about recursive definitions, properties that depend on iterating over data structures of unbounded size are hard to prove without explicit induction. This issue arises, for example, in structures with recursive insertion logic, such as the Replicated Growable Array, where determining where a new element belongs requires walking an unbounded number of existing elements, often leading to timeouts or inconclusive results.

Another limitation of automated verification is that it does not guarantee that the implemented conflict-resolution policy aligns with the intended semantics. Convergence alone cannot distinguish between different behaviors, such as add-wins or remove-wins. In addition, verification results may be unstable, as small, non-functional code changes can lead to unexpected verification failures. These limitations show that automation alone is insufficient for verifying modern CRDTs, motivating the use of deductive techniques that can reason about induction, enforce invariants, and provide stronger semantic guarantees.

1.3 Goals and Expected Contributions

The expected contributions of this work are threefold. First, it introduces a dedicated language for development of CRDTs, together with a source-to-source compiler that automatically derives both a VeriFx and a WhyML implementation from a single specification, removing the need to author and maintain two independent formalizations by hand. Second, it delivers a library of ten CRDTs verified through this pipeline, covering state-based and operation-based counters, sets, a generic map, and the Replicated Growable Array, together with the translation patterns required to generate well-integrated code for both tools. Third, an experimental evaluation compares the specification effort required from the programmer against the code generated for each target, and examines a case in which the two backends disagree, the Replicated Growable Array, whose Why3 proof succeeds where the fully automated VeriFx proof currently does not, illustrating the practical value of combining both verification paradigms.

Within this context, the specific goals of the dissertation are as follows:

- To design a single specification language capable of expressing both state-based (CvRDT) and operation-based (CmRDT) CRDTs.
- To formalize a source-to-source compilation pipeline that translates this language into both VeriFx and WhyML, hiding the syntactic and modelling complexity of each target from the programmer.
- To evaluate the resulting framework on CRDTs of increasing structural complexity, including recursive data structures, and characterize the cases in which automated

and deductive verification agree or diverge.

1.4 Report Structure

The remainder of this document is organized as follows:

- Chapter 2 presents the state of the art on consistency models, CRDT theory, and existing formal verification tools.
- Chapter 3 details the theoretical foundations required for this work, including the formalization of CRDTs, the logic underlying SMT solvers, and the principles of deductive verification with Why3.
- Chapter 4 presents the specification language introduced by the framework, its syntax, the CvRDT and CmRDT interfaces and the axioms attached to them, the constraints a module must satisfy to implement an interface, the directives that let a specification diverge between backends, and the type system underlying all of it.
- Chapter 5 describes how a validated specification compiles to both VeriF_x and Why3, the pattern each backend follows, how the compiler chooses between several generators depending on a CRDT's structure, and RGA as the case where every mechanism applies at once.
- Chapter 6 evaluates the framework across the ten CRDTs it specifies, comparing the manual effort of writing each specification against the code generated for each backend, and turning then to the proofs themselves, nine of the ten close automatically in both VeriF_x and Why3, one needs a single manual step in Why3, and RGA is the one CRDT where the two backends genuinely disagree, proved in Why3, rejected in VeriF_x, for reasons traced back to exactly how each backend compiles it.
- Chapter 7 closes the dissertation, summarising the problem addressed, the framework's design, and the evaluation results, and proposes extending the evaluation to further recursive or unbounded CRDTs as a direction for future work.

RELATED WORK

In this chapter, the relevant concepts and related work on consistency, replication, conflict resolution in distributed systems, and verification methods are presented. It starts by introducing consistency and replication models, clarifying the trade-offs between strong and weak consistency. The chapter then discusses how divergent replicas are handled, covering both ad-hoc conflict resolution strategies and principled approaches based on Conflict-free Replicated Data Types (CRDTs) [20]. Then, the synchronization models for CRDTs and the challenges that arise when composing and nesting replicated data structures are discussed. Finally, it addresses the role of formal verification in ensuring correctness, reviewing different verification approaches and the tools commonly used in practice, along with their capabilities and limitations.

2.1 Consistency models and Replication

Consistency models are commonly used to characterize the behavior of a system when multiple write and read operations are executed concurrently, defining the conditions under which different results can be observed. By establishing these rules, the model determines the level of consistency that applications can rely on. However, stronger consistency guarantees often come at the cost of reduced availability provided by the system, as established by the CAP theorem [8].

The CAP theorem states that a distributed system can not simultaneously provide strong consistency and high availability in the presence of network partitions. Since network partitions are unavoidable in practice, systems must adopt different trade-offs depending on the purpose of the system.

Under strong consistency assumptions, it is considered that once an operation completes, all replicas reflect a uniform state, ensuring that clients observe the most recent write regardless of the node accessed. Alternatively, approaches that prioritize availability relax these guarantees to maintain availability during failures.

2.1.1 Weak Consistency

Systems that adopt a weak consistency model prioritize high availability and low latency when responding to client requests. In this model, client operations are executed on a small subset of replicas, which execute the operation locally and reply to the client before the update is propagated to the remaining replicas. This propagation occurs asynchronously, avoiding the need for coordination with a majority of replicas, resulting in lower latency and higher availability. However, the relaxed coordination requirements among replicas lead to divergencies, where different replicas can return different values for the same read operation. While these systems may eventually converge, the temporary divergence between replicas increases the complexity of reasoning about system behavior, making application development and debugging more challenging [20].

Eventual Consistency is a weak consistency model that prioritizes availability over consistency guarantees. Under this model, replicas are allowed to diverge temporarily, with the only guarantee being that, in the absence of further updates, all replicas will eventually converge to a common state. During this convergence period, clients may observe out-of-date or inconsistent values across replicas.

To ensure convergence, eventual consistent systems rely on additional mechanisms for resolving conflicting updates.

Causal Consistency is one of the strongest weak consistency models that can be achieved while maintaining availability. This model requires the system to maintain causal dependencies between operations and guarantees that all clients observe causally related operations in an order that respects these dependencies.

Two operations are causally related when the execution of one can influence the execution of the other. For example, in a social networking application, if a user creates a post and subsequently deletes it, the delete operation is causally dependent on the create operation and must be observed after it. Operations that are not causally related are considered concurrent and may be observed in different orders by different replicas. With this enforced consistent ordering of causally related operations while allowing flexibility in the ordering of concurrent ones, causal consistency provides higher consistency guarantees than eventual consistency without sacrificing availability [4, 13].

Causal+ Consistency extends causal consistency by ensuring that replicas converge to the same state even in the presence of concurrent updates. Although causal consistency preserves the ordering of causally related operations, it allows concurrent operations to be applied in different orders, potentially leading to divergent replica states. Causal+ consistency overcomes this issue by enforcing a convergent conflict resolution strategy, in which concurrent operations are merged using associative and commutative functions, guaranteeing eventual convergence while preserving causal dependencies [23].

2.1.2 Strong Consistency

Systems that adopt a strong consistency model prioritize providing a single, well-defined view of the system state to all clients. In these systems, client operations are executed in a coordinated manner across replicas, ensuring that read operations always observe the effects of previously completed writes according to a well-specified ordering.

To enforce this guarantee, strong consistency models require tighter coordination among replicas, often involving synchronous communication and agreement protocols before operations are considered complete. While this coordination simplifies reasoning about system behavior and enables applications to rely on intuitive semantics, it typically comes at the cost of increased latency and reduced availability when compared to weakly consistent systems.

Linearizability is a strong consistency property that enforces a strict ordering of operations in replicated systems. It requires that each operation on a shared object appears to take effect at a single, indivisible point in time between its invocation and completion. Consequently, the outcome of all operations must be consistent with a global order that reflects the real-time order in which operations are issued by clients.

This property guarantees that once a write operation completes, its effects are immediately visible to all subsequent operations, independently of the replica they access.

Serializability is a consistency property that considers the execution of transactions, each composed of one or more operations that may access multiple objects. It guarantees that the execution of a set of concurrent transactions is equivalent to a serial execution, where transactions are totally ordered.

With this property, all replicas observe transactions in the same order. However, this order is not required to respect real-time constraints. Therefore, the serialization order may differ from the temporal order in which transactions are issued according to a global wall clock.

Sequential Consistency is a consistency property in which all clients observe operations as if they were executed in a single global order. This order is shared by all replicas, ensuring that every client perceives the same sequence of operations, even if this sequence does not correspond to the real-time order in which operations are issued.

While the global execution order does not need to respect real-time constraints, it must preserve the program order of each individual client or process. That is, if a process issues an operation *op1* before *op2*, all replicas must observe *op1* preceding *op2* in the global sequence.

Sequential consistency operates with individual operations. For this reason, systems that guarantee serializability also satisfy sequential consistency, whereas the opposite is not true. By enforcing a common operation order across replicas, sequential

consistency prevents clients from observing divergent states when accessing different replicas.

2.1.3 Strong Eventual Consistency

Strong Eventual Consistency characterizes a class of replication models in which replicas are allowed to execute updates independently and propagate them asynchronously, without relying on coordination or global ordering. The central guarantee of this model is that any replicas that have applied the same set of updates will deterministically converge to an identical state, regardless of the order in which those updates are delivered or executed [23].

This model is typically defined by three properties. *Eventual Delivery*, ensures that all updates are eventually propagated to all correct replicas. *Strong Convergence* guarantees that replicas that have applied the same set of updates reach an identical state. *Causal Consistency* preserves the ordering of causally related updates. Together, these properties prevent permanent divergence while avoiding the coordination overhead required by strongly consistent models.

Strong Eventual Consistency directly addresses the limitations of weakly consistent systems by eliminating non-deterministic conflict resolution, while retaining high availability and low latency under network partitions. In practice, strong eventual consistency is achieved through Conflict-free Replicated Data Types, whose design ensures that concurrent updates can be merged deterministically, enabling convergence without coordination [20].

2.2 Conflict Resolution Strategies

In practice, many replicated systems adopt weak consistency models in order to achieve high availability and low latency, allowing replicas to diverge temporarily as updates are executed independently and propagated asynchronously. While this design choice improves system responsiveness, it introduces the possibility of conflicting updates, making conflict resolution a fundamental requirement to obtain a consistent state.

Conflict resolution strategies define how concurrent or divergent updates are reconciled once replicas exchange their states or operations. Historically, these strategies have been broadly classified into ad-hoc approaches, which resolve conflicts using application-specific or heuristic rules, and principled approaches based in formal properties, such as those employed by Conflict-free Replicated Data Types. These approaches have a direct impact on system semantics, predictability, and scalability.

Consequently, conflict resolution is closely linked to the underlying replication and consistency model, influencing not only correctness guarantees but also the degree of coordination required and the overall system behavior under concurrency and failures.

2.2.1 Ad-hoc Strategies

Ad-hoc conflict resolution strategies resolve conflicts using simple, often application-specific rules, without providing formal guarantees of deterministic convergence in the presence of concurrent updates, as they may lead to undesirable behaviors.

Last-Writer-Wins is one of the most common conflict resolution policies. Under this strategy, concurrent updates are resolved by selecting the update with the most recent timestamp or highest value according to a predefined global ordering.

Although Last-Writer-Wins is simple to implement and introduces minimal coordination overhead, it is prone to lost updates, as concurrent modifications may overwrite each other without being merged. For example, in a replicated shopping cart, concurrent additions of different items by multiple users may result in only one of the updates being reflected in the final state, leading to unintended data loss [23].

Programmatic conflict resolution delegates the responsibility of resolving conflicts to the application logic. This approach was popularized by systems such as Amazon Dynamo [6], where replicas may return multiple conflicting versions of the same object to the client, which is then responsible for reconciling them and writing back a resolved version. While this strategy provides flexibility and allows applications to define domain-specific resolution semantics, it places a significant load on developers. Designing correct and efficient merge logic is often complex, particularly in the presence of high concurrency, and errors in conflict resolution may lead to inconsistent or unintended application states [11].

2.2.2 Conflict-free Replicated Data Types

Conflict-free Replicated Data Types emerge as a rigorous alternative for addressing replica divergence without the drawbacks associated with ad-hoc conflict resolution strategies. Rather than resolving conflicts through heuristic or application-specific rules, CRDTs embed conflict resolution directly into the data type design, ensuring predictable and well-defined behavior under concurrent updates.

A CRDT is an abstract data type designed to be replicated across multiple nodes, allowing replicas to perform local updates without requiring coordination. Convergence is ensured through mathematical principles, such as the commutativity and associativity of operations or the use of state-based merge functions defined over join-semilattices [23]. Therefore, any two replicas that have applied the same set of updates are guaranteed to reach an identical state deterministically, regardless of message ordering or network delays.

CRDTs include a variety of classes designed for different application needs, including counters, registers, sets, and graphs [17]. Some of these data types exhibit recursive structures, where operations may depend on nested or hierarchical elements, enabling the

design of more complex CRDTs that maintain convergence guarantees even in recursive scenarios [2].

In contrast to ad-hoc strategies, CRDTs ensure that all concurrent updates are reflected in the final state, avoiding unintended data loss. Since CRDTs do not rely on consensus protocols in the critical path of update execution, they exhibit high scalability and fault tolerance. Updates can be applied with low latency and propagated asynchronously, enabling systems based on CRDTs to remain available and responsive even in the presence of network partitions.

2.2.3 Hybrid and Tunable Policies

While standard CRDTs provide strong convergence guarantees, they typically enforce a fixed conflict resolution policy, such as add-wins or remove-wins semantics, which may not be suitable for all application scenarios. In practice, applications often require policies that depend on the context, for example, temporary disconnections may favor add-wins semantics to preserve availability, while explicit actions such as removals or bans may require remove-wins semantics to ensure they take effect. To address this limitation, hybrid and tunable approaches have been proposed [21], allowing conflict resolution semantics to be adapted for individual updates. Tunable CRDTs enable developers to select the desired resolution policy for each update, rather than committing to a single global policy.

2.3 Synchronization Models of CRDTs

The synchronization model defines the assumptions and guarantees required by a system to ensure the correct behavior of CRDTs in a replicated environment. In particular, it specifies how updates are propagated among replicas and under which conditions convergence is achieved, ensuring that all replicas that receive the same set of updates eventually reach an identical state. To this end, CRDTs rely on two fundamental synchronization models, state-based replication, in which replicas periodically exchange their full state, and operation-based replication, where individual updates are disseminated and applied at remote replicas [23].

2.3.1 Convergent Replicated Data Types

Convergent Replicated Data Types adopt a state-based synchronization model, in which replicas periodically propagate their entire local state to other replicas. Upon receiving a remote state, a replica integrates it with its own state through a deterministic merge function, ensuring that both replicas advance towards a common state. This approach decouples local updates from coordination, allowing replicas to apply operations independently and reconcile divergences through state exchange.

Convergence in CvRDTs is achieved by constraining how replica states evolve and how divergent states are reconciled. The set of possible states is partially ordered, allowing

replicas to compare states in terms of the information they contain. Local updates are required to be monotonic, meaning that applying an update to a state always produces a new state that is greater than or equal to the previous one. When replicas exchange state, reconciliation is performed by a deterministic merge function that computes the least upper bound of the two states, preserving all information observed by either replica. This merge operation is necessarily commutative, associative, and idempotent, ensuring deterministic convergence despite message reordering, duplication, or frequency of state exchanges.

From a systems perspective, CvRDTs impose minimal requirements on the underlying network. They tolerate message loss, duplication, and reordering, assuming only that communication is eventually reliable and that replicas can re-establish connectivity after partitions. This robustness makes CvRDTs particularly well-suited for highly unreliable networks, or environments subject to network partitions. However, this model may incur significant communication overhead, as entire states must be transmitted during synchronization, which can become inefficient for frequently updated objects.

2.3.2 Commutative Replicated Data Types

Commutative Replicated Data Types adopt an operation-based synchronization model, in which replicas propagate individual update operations rather than exchanging their full state. Each update is executed locally and disseminated to other replicas, allowing changes to be applied incrementally and avoiding the transmission of large state payloads.

When an update is issued, it is first processed at the origin replica, where any necessary context is used to produce an operation that can be safely applied at other replicas. This operation is then disseminated to the remaining replicas, where it is applied to the local state of each one. Therefore, replicas can generate updates independently, while ensuring that all replicas eventually apply the same operation and reach an identical state.

Convergence in CmRDTs relies on the commutativity of update operations. Since replicas may receive and apply operations in different orders, all operations that can be executed concurrently are required to commute, ensuring that the order in which updates are applied does not affect the final state. Once all replicas have received the same set of operations, they deterministically converge to an identical state, without requiring global coordination or synchronization.

Compared to state-based CRDTs, CmRDTs place stronger requirements on the underlying communication channel. Operations must be reliably delivered to all replicas, typically with causal ordering guarantees, to ensure that concurrent updates commute as expected and replicas do not diverge. While this increases the complexity of the communication layer, it significantly reduces synchronization overhead and makes CmRDTs more efficient for objects with large state or high update rates.

2.3.3 Composition and Nesting

While the properties of basic CRDTs are well understood, practical systems frequently require combining these primitives into more complex, hierarchical structures, such as maps containing nested objects, JSON documents, or directory trees. In these settings, simply nesting CRDTs is not sufficient, as the convergence guarantees of individual data types do not automatically extend to the composed structure. A challenge arises when concurrent operations affect different levels of the hierarchy, for instance, when an update modifies a nested element while a concurrent operation removes or replaces its parent.

These challenges have been studied in the context of replicated JSON documents [10], which are often regarded as a reference model for conflict resolution in deeply nested data. CRDT based approaches to JSON highlight that simple resolution policies, such as Last-Writer-Wins, are insufficient to preserve user intent in hierarchical structures, as they may discard meaningful concurrent updates. Consequently, specialized mechanisms are required to reconcile edits across multiple levels of the hierarchy while maintaining deterministic convergence.

To address these issues, several mechanisms for combining CRDTs have been proposed. One commonly adopted approach is recursive reset semantics, where removing an element from a collection implicitly resets or discards all nested state. More recent approaches generalize this idea by explicitly capturing the causal relationships between parent and child objects. In particular, nested operation-based CRDT models [2] introduce mechanisms that allow removals or resets to be applied selectively and recursively, based on causal information, ensuring that the semantics of the enclosing structure are preserved, providing deterministic convergence for complex, hierarchical CRDTs without introducing excessive metadata overhead.

2.4 Formal Verification

Formal verification of replicated data types is essential because it is difficult to reason about the many possible execution orders of concurrent operations in distributed systems. Subtle combinations of concurrent updates, message reordering, or partial failures can lead to unexpected behaviors that are hard to detect through testing alone. Therefore, informal reasoning or ad-hoc validation techniques are often insufficient to guarantee correctness [4].

Research in verification of replicated data types has evolved from manual, pen-and-paper proofs toward automated and deductive verification techniques. Early approaches relied heavily on human reasoning to establish convergence and correctness properties, while more recent work has focused on developing formal frameworks and tool-supported methods that provide stronger guarantees and better scalability [19].

2.4.1 Manual Verification

Early efforts to validate CRDTs relied primarily on manual, pen-and-paper proofs [23]. These proofs aimed to establish key properties such as convergence and correctness by reasoning about the abstract behavior of the data type under concurrency. From a theoretical perspective, this approach offers a high level of rigor, as it allows precise arguments to be constructed over well-defined formal models.

However, manual verification presents significant limitations. Constructing such proofs is time consuming and requires substantial expertise in formal methods, making the approach difficult to scale beyond a small number of data types. Moreover, these proofs often reason about an abstract specification of the algorithm, which may differ from its concrete implementation. Consequently, even when the specification is formally correct, errors may still occur in the implementation.

2.4.2 Automated Verification

To make formal verification more accessible to non-specialists, several automated verification approaches have been proposed, often based on high-level specification languages [9]. One such example is VeriFx [19], a framework that allows replicated data types to be implemented and automatically verified for Strong Eventual Consistency using SMT solvers such as Z3. By abstracting away low-level proof details, these tools significantly reduce the complexity of verifying replicated data structures and have been successfully applied to a wide range of basic CRDTs.

Despite these advances, the literature identifies important limitations in existing automated verification tools. In particular, handling recursive algorithms remains challenging, as demonstrated by the inability of VeriFx to verify data types such as the Replicated Growable Array, whose correctness depends on recursive insertion logic. Furthermore, the verification of more complex objects, especially those relying on version vectors, can be unstable in practice. For instance, small and semantically irrelevant changes to a specification, such as renaming variables, may cause the solver to fail or abort the proof process, highlighting limitations in robustness and predictability.

2.4.3 Deductive Verification

To overcome the limitations of purely automated verification tools, deductive verification frameworks provide an alternative approach. One such framework is Why3 [7], which is based on the WhyML [24] specification language and first-order logic. Why3 generates proof obligations that can be discharged by a variety of external theorem provers, allowing verification tasks to be expressed in a structured and modular way.

2.4.4 Invariants and Security

While the formal verification of CRDTs traditionally focuses on structural convergence, such as ensuring that merge operations form a join-semilattice, the correctness of a distributed application often depends on application-level invariants, including access control policies and other security-related constraints that extend beyond the data type itself.

This challenge has been studied in the context of distributed file systems operating under local-first assumptions [25], where security requirements are commonly expressed in terms of confidentiality, integrity, and availability. Previous work in this area has shown that conventional access control models, such as those inspired by POSIX, give rise to conflicts when replicas operate independently and synchronize asynchronously. These studies establish an impossibility result analogous to the CAP theorem, in systems that prioritize high availability and tolerate network partitions, it is not possible to simultaneously guarantee confidentiality, integrity, and full availability during conflict resolution, meaning that one of these properties must be relaxed when reconciling concurrent updates.

2.5 Verification Tools

Verification tools for Replicated Data Types differ in how they balance automation, expressiveness, and proof stability. Initial approaches were based on highly expressive interactive proof assistants, such as Isabelle/HOL [18], which offer strong guarantees but require substantial expertise and manual effort. More recent tools adopt SMT-based automation [19] to simplify the verification process at the cost of reduced expressiveness and, in some cases, stability.

2.5.1 VeriFx

VeriFx [19] represents a significant step toward making the formal verification of CRDTs accessible. It provides a high-level language, inspired by Scala, in which developers can implement replicated data types using functional abstractions, while automatically verifying properties such as Strong Eventual Consistency through SMT solving with Z3 [16].

A key strength of VeriFx lies in its ability to connect specification, verification, and deployment. Designs that are proven correct at the specification level can be translated into executable implementations in languages such as Scala and JavaScript, reducing the gap between formally verified models and practical systems.

Despite these advantages, VeriFx presents several fundamental limitations that restrict its applicability to more complex designs. The tool struggles with recursive algorithms, failing, for instance, to verify the Replicated Growable Array due to the recursive nature of its insertion logic, which exceeds the solver’s ability to reason about unbounded execution paths. In addition, VeriFx exhibits a pronounced sensitivity to syntactic changes, where minor modifications, such as renaming a variable, can cause previously successful proofs to

fail or vice versa. Finally, VeriF_x does not fully capture the semantic intent of certain conflict resolution policies, as it may prove convergence for a data type even when the verified semantics, such as remove-wins instead of add-wins, do not align with the programmer’s intended behavior.

In addition to these tool-specific limitations, SMT-based verification approaches face significant challenges when reasoning about nested and recursive CRDTs. In such cases, solvers may fail to converge within finite time, forcing developers to resort to manual reasoning techniques, such as structural induction, to establish correctness properties.

2.5.2 Why3

Why3 [7] is a platform for deductive program verification based on a first-order specification and programming language called WhyML [24]. Rather than relying on a single automated solver, Why3 serves as an intermediate verification layer that generates proof obligations and dispatches them to multiple external provers, such as Alt-Ergo [5], Z3 [16], and CVC [1].

Compared to purely SMT-based approaches, Why3 offers greater expressiveness and stability when reasoning about complex data structures. Its deductive verification model is particularly well-suited for nested or recursive definitions, which are difficult to handle using direct SMT encodings. In addition, Why3 enforces a static discipline over mutable state and aliasing, avoiding the need for intricate memory models and resulting in proof obligations that are easier to reason about and maintain.

As a consequence, Why3 provides a more flexible and controlled verification environment than fully automated tools such as VeriF_x. While this approach requires more user guidance and interaction, it enables the verification of a broader class of replicated data types [15] and offers a solid basis for addressing limitations related to recursion, modularity, and proof stability.

BACKGROUND

This chapter establishes the theoretical foundations essential for the work developed in this dissertation. It starts by presenting the formalization of Conflict-free Replicated Data Types (CRDTs) [20], detailing the synchronization models based on state and operations, as well as the role of causality in ensuring convergence. It then explores VeriFx, describing its approach to CRDT modeling and its automatic verification mechanism based on SMT solvers. Finally, it introduces the Why3 platform and the WhyML language [24], highlighting the support for deductive verification through explicit invariants and ghost code, which are fundamental for overcoming the limitations of purely automated approaches.

3.1 Formalization of CRDTs

The correctness of CRDTs is achieved through strong mathematical properties that ensure convergence across replicas, rather than through heavy coordination mechanisms. Regardless of message reordering, duplication, or temporary network failures, replicas that apply the same set of updates are guaranteed to reach an identical state. This section presents a formal view of CRDTs by characterizing the two fundamental synchronization models, the state-based model CvRDTs, and the operation-based model CmRDTs, as well as the notion of causality that supports their correctness guarantees.

3.1.1 State Based Convergence

State-based CRDTs ensure convergence by defining their state space as a join-semilattice [23]. Formally, a CvRDT is defined by a triple $(S, \leq, \amalg)$, where S denotes the set of possible states, $\leq$ is a partial order over S , and $\amalg : S \times S \rightarrow S$ is a merge operator that computes the least upper bound of two states.

To guarantee deterministic convergence, the merge operation must satisfy three algebraic properties for all states $s_1, s_2, s_3 \in S$, to ensure that merging replicas yields the same result regardless of message ordering, duplication, or repetition. These properties are: associativity $(s_1 \amalg s_2) \amalg s_3 = s_1 \amalg (s_2 \amalg s_3)$, commutativity $s_1 \amalg s_2 = s_2 \amalg s_1$, and idempotence, $s_1 \amalg s_1 = s_1$.

In addition, all local update operations must be monotonic with respect to the partial order $\leq$. That is, if a state s is updated by an operation u , producing a new state s' , it is required that $s \leq s'$. This monotonicity property ensures that replicas always progress within the lattice, preventing regressions that could otherwise lead to divergence in asynchronous systems. The function $compare(x, y)$, used in practical implementations, corresponds to the partial order relation, which returns true if $x \leq y$. This relation is then used to define state equivalence, where two states are considered equivalent if and only if $x \leq y$ and $y \leq x$.

3.1.2 Operation Based Convergence

Operation-based CRDTs achieve convergence by disseminating update operations rather than exchanging full states. In this model, an update is first processed at the source replica, where its preconditions are checked and the corresponding operation is generated. This operation is then propagated through the system and applied to the local state of all replicas, ensuring that each replica eventually executes the same set of updates.

A sufficient condition for convergence in a CmRDT is that all concurrent operations commute [23]. Given two operations f and g that are concurrent, applying them in any order must yield the same resulting state $S \cdot f \cdot g \equiv S \cdot g \cdot f$. This commutativity property allows the replication system to relax the delivery order of concurrent messages, enforcing causal delivery, and ensuring that operations are applied in an order consistent with the happens-before relation, while allowing concurrent updates to be delivered in any order.

3.1.3 Pure Operation-Based CRDTs

Traditional operation-based CRDTs enable the generation of an update to inspect the current state of a replica to produce the message that will be disseminated. Pure operation-based CRDTs [2] adopt a more restrictive approach, in which the propagated message contains only the original operation and its arguments, without including any additional metadata derived from the replica state.

In this model, the state of a replica is represented as a partially ordered log that records the applied operations, together with their causal ordering. Rather than computing the state directly, replicas derive the current state from this log.

To control the size of the log, this model relies on two concepts, the first being causal redundancy, where a newly delivered operation makes the effects of earlier operations no longer relevant. For example, a remove operation can remove the effect of a previous add of the same element, allowing the earlier operation to be safely ignored. Identifying such cases enables the system to discard redundant operations from the log without altering the resulting state.

The second concept is causal stability. When it is known that all replicas have observed an operation and that no concurrent operations remain, the causal information associated

with that operation is no longer needed. This allows replicas to discard causal metadata while still guaranteeing deterministic convergence.

3.1.4 Causality and Logical Clocks

In distributed systems without centralized coordination, the notion of time is defined through the happens-before relation, originally introduced by Lamport [12]. This relation plays a central role in CRDTs, as it allows the system to distinguish between sequential and concurrent events, which is essential for data types whose semantics depend on concurrency, such as add-wins sets or multi-value registers.

Causality establishes that an event a occurs before an event b when a and b take place at the same replica and a precedes b , for example, when a corresponds to the sending of a message and b to the reception of that message, or when the ordering follows transitively from other events. If neither $a \rightarrow b$ nor $b \rightarrow a$ holds, the two events are considered concurrent.

To represent causality, systems commonly rely on version vectors. A version vector associates each replica identifier with an integer counter that records how many updates originated at that replica and are reflected in the current state. When a replica performs a local update, it increments its own counter, and version vectors are merged by taking, for each replica, the maximum of the corresponding counters. This allows causal histories of states or operations to be compared, a version vector is considered causally ahead of another if, for every replica, its counter is greater than or equal to the corresponding counter in the other vector. When neither vector is causally ahead of the other, the associated states are concurrent, indicating a conflict that must be resolved by the CRDT, for example by merging values or applying a predefined resolution policy.

3.2 Modeling and Verification in VeriFx

VeriFx [19] is a high-level programming language designed specifically for the implementation and formal verification of Replicated Data Types. It follows a functional and object-oriented style, restricting programmers to immutable collections and a set of logical constructs, allowing VeriFx programs to be translated into SMT formulas and automatically verified using the Z3 solver [16].

3.2.1 RDT Modeling

In VeriFx, Replicated Data Types are modeled by extending traits provided by the standard library. These traits act as partially implemented interfaces that define the structural and semantic requirements that a data type must satisfy in order to be verified. By constraining implementations to follow these abstractions, VeriFx ensures that the resulting models are suitable for automatic verification of convergence and correctness.

State-based CRDTs are modeled by extending the `CvRDT[T]` trait. This interface requires the implementation of a `merge(that: T): T` method, which defines how two replica states are combined by computing their least upper bound, and a `compare(that: T): Boolean` method, which encodes the partial order over states and is used to reason about equivalence and monotonicity.

Operation-based CRDTs are modeled by extending the `CmRDT[Op, Msg, T]` trait. In this case, updates are represented as operations that are transformed into messages for dissemination and later applied at all replicas. The `prepare(op: Op): Msg` method generates the message corresponding to an update without modifying the local state, while the `effect(msg: Msg): T` method applies a received message to produce the updated state, allowing VeriFx to reason explicitly about the propagation and application of operations when verifying convergence.

3.2.2 Automated Verification

The distinctive feature of VeriFx lies in its support for automated verification directly integrated into the language through the proof construct. During compilation, the VeriFx program and associated proof definitions are translated into logical formulas understood by the Z3 SMT solver. The solver then attempts to determine whether these formulas can be violated by any admissible execution, searching for counterexamples in the space of possible states or operation histories. If no counterexample is found, the specified property is considered to hold for all executions admitted by the model.

Proof Construction defines how correctness properties are expressed and checked in VeriFx. A proof is written as a function without parameters whose body consists of a Boolean expression intended to be universally true. Instead of constructing a proof manually, the verification process attempts to falsify this expression by identifying a concrete execution that contradicts the stated property. This approach, based on counterexamples, enables fully automated reasoning while providing strong guarantees when verification succeeds. Proofs are evaluated over all states and executions that satisfy the constraints imposed by the data type definition, the replication model, and the admissibility conditions encoded in the program.

Convergence Verification focuses on establishing Strong Eventual Consistency through reusable proof abstractions provided by the VeriFx library. For state-based CRDTs, the verifier generates proof obligations that check whether the merge function defined by the user induces a join-semilattice over the set of reachable and compatible states, ensuring idempotence, commutativity, and associativity. For operation-based CRDTs, the verification focuses on ensuring that all concurrent operations commute, the verification engine explores executions in which admissible operations are applied in different orders and checks that the resulting states are equivalent. This allows

convergence properties to be validated systematically across all possible execution orderings admitted by the replication model.

3.3 Deduction and Specification in Why3

Why3 [7] is a platform for deductive program verification that provides a rich specification and programming language, known as WhyML. Instead of relying on a single automated solver, Why3 acts as an intermediate verification framework it translates specified programs into verification conditions, which are then submitted to a range of external provers, including both automated solvers, such as Z3 [16], CVC [1], and Alt-Ergo [5], and interactive proof assistants, such as Coq [3], allowing users to combine automation with manual reasoning when required.

3.3.1 WhyML

WhyML is designed to act simultaneously as a programming language and as a language for logical specification, allowing executable code and formal properties to be expressed within a single framework, and enabling specifications to be written close to the code described, while still supporting rigorous reasoning by automated and interactive provers. Syntactically, WhyML is close to languages such as OCaml, but enforces some restrictions to ensure that the verification conditions it generates remain manageable for provers and understandable to users.

In particular, WhyML is based on first-order logic, although it supports polymorphism and recursive algebraic data types, reasoning is restricted to first-order constructs, which keeps proofs manageable for automated provers. Higher-order functions are therefore excluded from the logic, even though nested function definitions remain available at programming level.

Another defining aspect of WhyML is the absence of a complex memory model. Rather than explicitly modeling the heap, it enforces static control over aliasing. Each mutable reference has a finite and statically known set of names, and recursive data types are restricted from containing mutable components, preventing the formation of data structures with complex sharing patterns that would complicate logical reasoning.

Mutable state is encapsulated within structures containing mutable fields, allowing the type system to track updates and ensure that state changes are explicitly reflected in the corresponding proof obligations.

3.3.2 Ghost Code and Invariants

Verification in Why3 is based on specifications of program behavior, expressed through preconditions using `requires`, postconditions using `ensures`, and invariants. These specifications are used in functions where they describe the requirements of a method or

function, and the expected behavior upon termination for preconditions and postconditions, respectively. In addition, loops and recursive functions must be included with invariants to capture correctness properties that hold throughout execution, and with variants to establish termination. For the verification of more complex algorithms, such as CRDTs, Why3 provides support for ghost code. Ghost code consists of auxiliary computations and data that are introduced to support formal reasoning and are not taken into account when producing executable code. Such code is constrained so that it cannot influence the control flow of the actual program or modify non-ghost state, ensuring that verification elements do not affect program semantics.

3.3.3 Deductive Proofs

In Why3, the verification process is based on the computation of weakest preconditions to generate proof obligations in the form of logical formulas from a program and its specifications. If these obligations are successfully verified, they guarantee that the program satisfies its specification and is free from certain runtime errors, such as division by zero or out-of-bounds array accesses.

A key advantage of Why3 over approaches that rely solely on SMT solving is its ability to reason about complex inductive and recursive definitions through the integration of interactive provers. This increases robustness and expressiveness when verifying nested or recursive data structures, where fully automated solvers often fail due to timeouts or limitations in handling induction.

LANGUAGE TO SPECIFY CRDTs

Chapters 2 and 3 established that automated and deductive verification address CRDT correctness from different angles, and that neither is sufficient on its own. The framework at the centre of this dissertation reconciles the two not by translating one tool’s artifacts into the other’s, but by introducing a dedicated language in which a CRDT is specified once, and from which both a VeriFx implementation and a Why3 specification are derived automatically.

Five sections follow, covering both the language itself and the constraints the compiler enforces on any specification written in it. The first, Section 4.1, covers the concrete syntax used to declare a CRDT, its state, its functions, and how these are grouped into a module. Section 4.2 then turns to how the `CvRDT` and `CmRDT` interfaces referenced by every module are themselves declared, together with the algebraic axioms that can be associated with them. Sections 4.3 and 4.4 describe two complementary sets of restrictions: first, the conditions a module’s declarations must satisfy to be accepted as an implementation of its interface; and second, the directives, `[@vfx]`, `[@proof]`, and `variant(...)`, that let a specification diverge, in controlled ways, between the VeriFx and Why3 backends. The chapter closes, in Section 4.5, with a complete description of the type system introduced informally in the sections before it.

4.1 Syntax

A specification in the framework’s language is built around two complementary constructs: interfaces, which state the contract an implementation must satisfy, its types, functions, and the algebraic properties expected of them, and modules, which provide a concrete implementation of one such contract for a specific CRDT. This separation is deliberate as it lets the compiler validate a module purely by checking it against the shape declared by its interface, Section 4.3, independently of how the module’s logic is eventually compiled into VeriFx or WhyML.

A module begins with a header of the form:

```
module <Name> : <Interface>
  ...
end
```

which names the CRDT being defined and states, via the `:` after the name, which interface it claims to implement, for instance, `module GCounter_CRDT : CvRDT`. This single line is what lets the backend decide, later in the pipeline, whether the module should be compiled along the state-based (CvRDT) or operation-based (CmRDT) path, without the programmer having to state this intent anywhere else.

Inside a module, type declarations introduce the vocabulary used by the rest of the definition. A concrete type is introduced with `type <name> = <type expression>`, most commonly used to define the module’s own state representation, `type payload = ...`. A type may also be declared without a body, `type <name>`, which introduces it as a generic parameter rather than a concrete definition. GSet declares `type elem` before using it in `type payload = set elem`, leaving the element type unconstrained so that the resulting CRDT can later be instantiated for any type. The type expressions available on the right-hand side of a type declaration include the primitive types `integer` and `boolean`, the collection types `map<K, V>` and `set T`, tuples `(T1, T2)`, records `{ field1: T1, field2: T2, ... }`, and variants, written either as a plain enumeration `A | B | C` or, when a case needs to carry data, as `A: T1 | B: T2`, Section 4.5.

Function declarations follow the keyword `val`, and share a single general shape:

```
val <name> (<parameters>) : <return type> = <body>
```

Parameters sharing a type can be grouped under a single annotation, as in `(a b: payload)`. The same pattern extends to any number of parameters, which keeps functions whose parameters repeat a type, `merge`, `compare`, and `equals`, among others, concise to declare. When a function takes no parameters, as is the case for the constructor `create`, the parentheses are omitted entirely, and the declaration reduces to `val create : <return type> = <body>`. A function body may optionally be preceded by a `requires { <condition> }` clause, placed between the return type and the body itself. This expresses a precondition on the function’s inputs, Section 4.4.

To make this concrete, Listing 4.1 gives the complete definition of `GCounter_CRDT`, the running example for the rest of this chapter.

```

module GCounter_CRDT : CvRDT
  type payload = integer

  val create : payload = { payload : 0 }

  val merge (a b: payload) : payload =
    max(a, b)

  val compare (a b: payload) : boolean =
    a == b

  val equals (a b: payload) : boolean =
    compare(a, b) && compare(b, a)

  val increment (v: integer, a: payload) : payload =
    requires { v > 0 }
    a + v
end

```

Listing 4.1: GCounter_CRDT module

One detail in this listing is easy to miss but worth explaining, since nothing in the syntax itself points to it. Although `payload` is declared as a plain `integer`, the body of `create` is written as a record literal, `{ payload : 0 }`, rather than simply `0`. This is not an inconsistency in the language but a direct consequence of how every module’s runtime state is represented internally. Regardless of whether a CRDT’s `payload` is itself a record, as it is for `PNCounter_CRDT` or `TwoPSet`, whose `payload` already has named fields, the framework always wraps a module’s state in an implicit record with a `payload` field, mirroring the two-module structure (`type t = { payload: ... }`) that the Why3 backend generates for every CRDT Chapter 5. Writing `create` as a record literal is therefore not an arbitrary convention but a syntactic consequence of this internal representation. It is the only place in a module where the shape of the compiler’s internal state representation surfaces in the code the programmer writes. Every other function operates on `payload` directly, with no need to mention this wrapper.

4.2 The CvRDT and CmRDT Interfaces

Every module in the language is written to satisfy one of two interfaces, `CvRDT` for state-based CRDTs and `CmRDT` for operation-based ones. Both are regular interfaces in the sense of Section 4.1. Nothing in the compiler treats the identifiers `CvRDT` or `CmRDT` specially during parsing. What is specific to these two, besides being the only two included in the framework, is that the compiler checks any interface carrying either of these exact names for a fixed minimum signature, and that the backend inspects which of the two a module implements to decide how to compile it, Section 4.3.

4.2.1 Interface Declarations

An interface is declared with:

```
interface <Name>[@proof]
  ...
end
```

The `[@proof]` tag is optional and related only to VeriFx, and its presence decides whether the generated proof trait for this interface includes the base convergence proof, the structural `is_a_CvRDT/is_a_CmRDT` obligation establishing the module as a genuine join-semilattice, or, for `CmRDT`, that its operations commute. This is the same proof defined in VeriFx’s own practical CRDT library. The framework’s generated `CvRDTProof/CmRDTProof` traits contain the same proof, so a module proven correct through this pipeline carries the same guarantees as one written directly against VeriFx’s own traits. Between `interface` and `end`, only two kinds of declarations are allowed: types, written `type <name>` with no body, and function signatures, written `val <name> (<parameters>) : <return type>` with no body either. An interface states what a conforming module must provide. Listing 4.2 gives the `CvRDT` and `CmRDT` interfaces exactly as declared in the framework:

```
interface CvRDT[@proof]
  type payload

  val create () : payload
  val merge (a b: payload) : payload
  val compare (a b: payload) : boolean
  val equals (a b: payload) : boolean

  axiom commutative(merge)
  axiom idempotent(merge)
  axiom associative(merge)
  axiom equivalence(compare)
end

interface CmRDT[@proof]
  type payload
  type operation

  val create () : payload
  val execute (op: operation, state: payload) : payload
  val compare (a b: payload) : boolean
  val equals (a b: payload) : boolean

  axiom op_commutative(execute)
end
```

Listing 4.2: The `CvRDT` and `CmRDT` interfaces

The two interfaces share almost the same shape. Both declare a payload type, a `create` with no parameters, and `compare/equals`. They diverge in exactly the operation each was designed around. `CvRDT` exposes `merge`, taking two states and returning their combination, the same join operation described in Section 3.1.1, now expressed as the signature every `CvRDT` module has to provide. `CMRDT` instead declares `type operation` and a single `execute`, taking an operation and the current state and returning the state after that operation has been applied. `VeriFx` presents this behavior splitting it into a `prepare` step that turns an operation into a message and an `effect` step that applies a received message to the state, Section 3.2.1. The language expresses this as a single function instead. A specification simply states the result of executing an operation on a state, and it is the `VeriFx` backend’s job to derive the `prepare/effect` split from it, Chapter 5. `Why3`, in turn, uses `execute` directly, with no equivalent split. The result is that a single `execute` definition is enough to target both backends, at the cost of that definition needing to be split apart again later, invisibly to the programmer, whenever the `VeriFx` backend needs it.

4.2.2 Axioms

Alongside its types and functions, an interface can declare axioms, properties that every module implementing the interface is expected to satisfy. An axiom can be declared in one of two ways. The first names one of five built-in properties:

```
axiom <kind>(<function>)
```

where `<function>` is the function the property applies to. The compiler turns each of these directly into automated proof obligations for `VeriFx`, Chapter 5, which is why this particular form is restricted to a fixed vocabulary rather than left open: it recognises exactly the five properties below, and rejects any other name:

- `commutative(merge)`: merging `x` with `y` produces the same result as merging `y` with `x`.
- `idempotent(merge)`: merging a state with itself leaves it unchanged.
- `associative(merge)`: merging three states gives the same result regardless of how the merges are grouped.
- `equivalence(compare)`: the `equals` method derived from `compare`, Section 4.1, agrees with the notion of equality between two states. Without this, `compare` could describe some relation weaker than genuine equivalence, and `equals` would no longer identify the same states `==` does.
- `op_commutative(execute)`: applying two operations to the same state in either order leaves the state in an equal, reachable result, the operation-based counterpart of `commutative` for `merge`.

Each of these compiles, for VeriF_x, into a fixed proof template built around the named function. For instance, `commutative(merge)` always produces a proof stating that `x.merge(y)` and `y.merge(x)` are equal and reachable for any two reachable, compatible `x` and `y`. This is why the set of properties is closed rather than extensible. Each one corresponds to a hand-written proof shape in the backend, and allowing an arbitrary property name would mean admitting a proof the backend has no template for.

The second form lets the programmer state a property the five built-in properties do not cover, by writing the condition directly:

```
axiom <name>: <boolean expression>
```

which attaches an arbitrary boolean formula, written using the same expression syntax as function bodies, directly to the interface under a name of the programmer's choosing. This form exists in the grammar and is fully supported by the type checker, but none of the ten CRDTs implemented so far has needed it. The five built-in properties have so far been sufficient to state every property these specifications require.

The five built-in properties can also appear without this closing on an interface at all, directly attached to a function declared inside a module rather than an interface, when that function has no body of its own:

```
val <n> (<parameters>) : <return type>
  axiom <kind>
  axiom <kind>
  ...
```

`NestedMap`, Appendix E, Listing E.5, uses exactly this for `merge_values` and `compare_values`, two functions it declares but never defines, only assumes satisfy `commutative`, `idempotent`, and `associative` for the former, and `equivalence` for the latter. This is what lets `NestedMap` stay generic over any value type: rather than committing to one concrete way of merging values, it states the properties a merge for that value type would need to have, and leaves the definition itself to whichever value type the module is eventually used with. The function name is omitted here, unlike in an interface's `axiom <kind>(<function>)`, because only one function is in scope for it to describe. Internally, both compile to the exact same representation, they differ only in how they are written.

4.3 Meeting an Interface's Requirements

A module is only accepted by the compiler once it has been checked against the interface named in its header. This checking happens in two layers. The first applies to any interface, and simply confirms that a module provides everything the interface declares. The second is stricter, but only applies to the two interfaces the framework actually ships with. Naming an interface `CvRDT` or `CmRDT` carries an additional obligation of its own.

4.3.1 Validating Modules

For every type the interface declares, the module must declare a type of the same name. For every function signature the interface declares, the module must provide a function of the same name, with the same number of parameters and a compatible return type. These checks are independent of one another and fail the moment they are not met, each with a message naming exactly what is missing or mismatched: Module '%s' missing type '%s' present in interface '%s'. for a missing type, Module '%s' missing function '%s'. when a function is absent altogether, Function '%s' has wrong number of arguments. for an arity mismatch, and Function '%s' return type does not respect the interface's. when the return type disagrees.

The second layer applies only to interfaces named exactly `CvRDT` or `CmRDT`, and is checked once, when the interface itself is declared, rather than separately for every module that later implements it. An interface named `CvRDT` must declare at least `create`, `merge`, `compare`, and `equals` among its functions. One named `CmRDT` must declare at least `create`, `execute`, `compare`, and `equals`. An interface with either name that omits one of these is rejected outright, with Interface '%s' must declare at least %s. naming what was expected.

A custom axiom, Section 4.2, declared on an interface is also carried over to every module that implements it. When a module is checked, each axiom `<name>: <expression>` on its interface is type-checked a second time, this time against the module's own concrete payload type, and the resulting formula is added to the module as an assumption. A module therefore never needs to restate a custom axiom itself. Satisfying the interface is enough for the axiom to apply to it.

4.3.2 State Representation and the payload/t Distinction

The return-type check described above allows for one exception. A module may declare its own type `t`, in addition to `payload`, to represent its state internally, as `VGCounter` and `UWMap` do. When it does, its functions are allowed to return `t` in place of `payload` and still satisfy the interface. Without this allowance, a module that needed a richer internal representation than `payload` alone, a vector indexed by replica for `VGCounter`, or a pair of maps tracking added and removed elements for `UWMap`, would have no way to express that representation without also changing what `payload` itself means to the outside world, and every other module composing it would have to change along with it.

This same mechanism also explains how one CRDT can be built out of another. Whenever a module declares a function literally named `create`, whether it uses type `t` or works with `payload` directly, the compiler automatically registers two functions other modules can call on it from outside, `<Module>.create` and `<Module>.get_payload`. `PNCounter_CRDT` relies on exactly this when it is built out of two `GCounter_CRDT` instances. Its own `create` calls `GCounter_CRDT.create` twice, once for each of its two counters, and its `merge` reads their values back out through `GCounter_CRDT.get_payload`. `PNCounter_CRDT` never

needs to know that `GCounter_CRDT`'s payload happens to be a plain integer rather than something more elaborate. It only ever sees `GCounter_CRDT` through these two functions. This is, in the end, why composing CRDTs in the language works at all. Every module with a `create` carries this pair of functions along with it automatically, without the programmer composing two CRDTs ever having to ask for them.

4.4 Backend-Specific Directives

Most of a specification compiles the same way for both backends. Neither backend needs to know the other one exists. A few things do not work this way though, and the language has four directives for them. `[@proof]`, on an interface, was already covered in Section 4.2, since it belongs to the interface as a whole rather than to one declaration inside it. This section covers the other three: `[@vfx]` on a function, `[@vfx class: ...]` on a type, and the `[@vfx]` version of `requires`. It also covers `variant(...)`, which is not tied to VeriFx in the same way, but fits here for a reason explained in Section 4.4.4.

4.4.1 VeriFx Functions

A function declaration can carry a simple `[@vfx]` tag, placed right after its parameters:

```
val <name> (<parameters>) [@vfx] : <return type> = <body>
```

A function tagged this way only exists for VeriFx, and Why3 does not see it, as if it had never been declared. This is needed because not every function a CRDT uses is something Why3's proof needs, some functions exist solely to support VeriFx's own proof machinery. `OpTwoPSet`, for example, declares `preRemove`, `enabledSrc`, `forall_op`, and `filter` this way. These help VeriFx prove convergence for a set-based `CmRDT`, while remaining unused on the Why3 side of the same specification. Five names are treated specially when tagged like this: `enabledSrc`, `enabledDown`, `compatibleS`, `reachable`, and `compatible`. These are the same methods `CvRDT` and `CmRDT` already give a default implementation for, Section 3.2.1, so a module that supplies one of them is overriding that default, and the generated VeriFx code writes `override` instead of a plain `def`.

4.4.2 Shaping the VeriFx Representation

The same `[@vfx ...]` syntax can also attach to a type declaration, where it can ask the VeriFx backend for one of three different things, depending on what follows `[@vfx`.

The first extracts the type into its own standalone class:

```
type <n> = { ... } [@vfx class: <fn1>, <fn2>, ...]
```

RGA, Appendix E, Listing E.10, needs a Lamport clock to order concurrent insertions, so it declares type `lamport_clock`, along with functions like `smaller_clock`, `increment_clock`, and `sync`, each marked `[@vfx]` as in the previous subsection. On their own, these would just become VeriFx-only functions on the RGA class. The `[@vfx class: ...]` directive on `lamport_clock` changes that: it pulls the five named functions out into their own `LamportClock` class. This way the generated VeriFx code models a Lamport clock the way someone writing VeriFx directly would, as its own type with its own methods, rather than as several functions defined directly on RGA. What the compiler requires is the `class` part, so `[@class: ...]` alone works exactly the same.

The second overrides the VeriFx type the backend would otherwise derive for a field, replacing it with whatever type expression follows `[@vfx]`:

```
type t = { <field>: <type expression>, ... } [@vfx <VeriFx type>]
```

`VGCounter`, Appendix E, Listing E.3, represents its state as type `t = { array: map <integer, integer> }`, a map from replica number to that replica's own count. Translated automatically, this would become a generic VeriFx `Map[Int, Int]`. Tagging the declaration `[@vfx Vector[Int]]` instead tells the backend to represent `array` as a `Vector[Int]`, matching how a vector-based counter is normally modelled directly in VeriFx, and giving access to vector-specific operations, such as `.zip` and `.map`, used later when compiling merge and compare for this CRDT, Chapter 5.

The third reuses an existing CRDT module's own VeriFx class for a field, instead of deriving a new one:

```
type payload = { <field>: <type expression>, ... } [@vfx <Module>: <field>]
```

RGA's `payload` carries an added and a removed set of elements, the exact shape already defined by `OpTwoPSet`, Appendix E, Listing E.8. Rather than deriving a second, unrelated representation for the same pair of sets, `[@vfx OpTwoPSet: vertices]` tells the backend to represent them as an instance of `OpTwoPSet`'s own generated VeriFx class, exposed on RGA under the field name `vertices`.

4.4.3 VeriFx Preconditions

A `requires` clause, Section 4.1, can also be marked `[@vfx]`:

```
requires { <condition> } [@vfx]
```

On the VeriFx side this behaves exactly like an ordinary `requires`, becoming the same `if/else` guard around the function body. On the Why3 side it disappears completely, no contract is generated at all, unlike an ordinary `requires`, which becomes a real Why3 precondition. RGA does this for `successor`, whose precondition, `lookup(e, a)`, is only declared `[@vfx]`. This lets VeriFx narrow down the states its solver explores, without needing Why3 to prove a precondition that is not actually necessary for the rest of the specification.

4.4.4 Proving Termination with variant

A recursive function can be given a termination measure with `variant(...)`, placed after its return type:

```
val <name> (<parameters>) : <return type>  
  variant (<expression>, ...)  
  = <body>
```

Why3 needs this to prove that a recursive function actually terminates: the expression given has to strictly decrease, in a way that cannot decrease forever, on every recursive call. VGCounter's value computation uses `variant (size - replica)` for this. VeriFx has nothing like this requirement, a `variant` clause simply does not affect what gets generated for it.

For a function like VGCounter's, this is enough, since the decreasing quantity is already a parameter with a clear meaning. RGA's `find_insert_position` needs more than that. Nothing in its own parameters already decreases as it walks along the structure, so the specification adds one just for this purpose, an explicit `fuel` parameter, decremented by one on every recursive call, with `variant (fuel)`:

```
val find_insert_position (stamp: lamport_clock, e left right: elem,  
                        a: payload, fuel: integer) : payload  
  
  variant (fuel)  
  =  
    if fuel <= 0 then  
      ...  
    else if smaller_clock(stamp, right.timestamp) then  
      find_insert_position(stamp, e, right, successor(right, a), a,  
                          fuel - 1)  
    else  
      ...
```

Why3 needs the `if fuel <= 0 then ...` branch as a fallback for when `fuel` reaches zero, even though it is not meant to run in practice. `fuel` itself has no meaning of its own beyond giving Why3 something it can prove decreases. The backend recognises this exact pattern, a parameter that only counts down inside an `if <param> <= 0 then ... else if ... then recurse(..., <param> - 1) else ...` shape, and removes it completely when compiling for VeriF_x. The `fuel` parameter is removed, along with the base case, and only the `else if/else` branches are left, as an ordinary recursive function:

```
private def find_insert_position(stamp: LamportClock, e: Elem[V], left: Elem[V],
                                right: Elem[V]): RGA[V] = {
  if (stamp.smallerClock(right.timestamp)) {
    this.find_insert_position(stamp, e, right, this.successor(right))
  } else {
    val newClock = if (this.clock.smallerClock(e.timestamp)) {
      e.timestamp
    } else {
      this.clock
    }
    new RGA(newClock, this.vertices.add(e), this.edges.add(e, right).add(left, e),
            this.bottom_value)
  }
}
```

Listing 4.3: `find_insert_position`, as generated for VeriFx

Without this, `fuel` would have to be present on the VeriFx side too: an argument with no purpose there, one the programmer calling `find_insert_position` would have to supply some value for on every call, just to satisfy a termination requirement that belongs only to Why3.

4.5 Type System

Section 4.1 introduced the type expressions available in the language, integer, boolean, set, map, tuples, records, and variants, without describing how the compiler checks them. This section addresses that gap, how a type written by the programmer is resolved into an internal representation the compiler can compare and reason about, how expressions are checked against that representation, and how the language determines a type in the one case where it cannot be read directly from the syntax, calls to `set.*` and `map.*`.

4.5.1 Types

Internally, every type expression the programmer writes is resolved into one of a small, fixed set of internal representations. `integer` and `boolean` become two fixed built-in types. A named type that is not one of these two is resolved differently depending on what it refers to, a type declared without a body, such as `GSet`'s type `elem`, becomes an abstract type, possessing no structure beyond its own name, while a type declared with a body, such as type `payload = ...`, is resolved to the structure that appears on its right-hand side, and the name itself is retained as an alias pointing to that structure. `map<K, V>` and `set T` become structured types parameterised by the types of their keys, values, or elements, themselves resolved the same way, recursively. Tuples, records, and variants follow the same pattern, `(T1, T2)` becomes a pair of resolved types, `{ field1: T1, ... }` becomes a list of field names paired with their resolved types, and a variant becomes either a plain

list of constructor names, for $A \mid B \mid C$, or a list of constructor names each paired with the type of data it carries, for $A: T1 \mid B: T2$.

Because a name such as `payload` is only an alias, comparing two types for compatibility cannot stop at the name. Two types can only be meaningfully compared once every alias they contain has been resolved to its underlying structure. The compiler performs this resolution on demand, wherever two types need to be compared, rather than once up front, an alias is expanded precisely when a given check requires looking past it. This is also why an abstract type such as `elem` behaves differently from every other type, it has no structure to expand into, and is therefore equal only to itself.

4.5.2 Typing Expressions

Every expression within a function body is type-checked against the types visible at that point in the program, the parameters in scope, the functions and records declared so far, and, where applicable, the type expected of the expression by its surrounding context. Checking an expression produces both its type and an internal, fully-typed representation of it, the same representation the backend later compiles from, leaving no aspect of an expression's meaning implicit beyond this point.

Most of the rules are what a reader familiar with a typed language would expect. Arithmetic operators require both sides to be integer and produce an integer. The ordering comparisons, `<` and `<=` among them, impose the same requirement. Equality, `==`, requires only that both sides resolve to the same type once aliases are expanded. The boolean connectives require both sides to be boolean. An `if` requires its condition to be boolean and its two branches to agree on a single type, which becomes the type of the `if` expression as a whole. A function call, whether to a value declared within the same module or to one exposed by another module through the `create/get_payload` mechanism described in Section 4.3, is checked for both the correct number of arguments and the correct type of each.

A `match` expression corresponds to what other functional languages term pattern matching, and can match on more than one expression at once, as RGA's `op_commutative` lemma does when it matches on a pair of operations together, `op1` and `op2`, to establish commutativity case by case. Each branch's pattern is checked against the structure of the variant type it matches, a constructor with no arguments, such as `Inc`, for a variant declared as a simple enumeration, or a constructor applied to one variable for each piece of data it carries, such as `Add v`, for a variant declared with $A: T1 \mid B: T2$. All branches are required to return the same type, determined from the first branch checked.

Constructing a record is the one construct whose type is not stated anywhere within the expression itself. The literal `{ field1: e1, field2: e2 }` names no type, only fields. The compiler resolves this by searching every record type declared throughout the specification for the one whose field names match exactly, and using that record's field types to check each of the given field expressions in turn. This matching is based purely on

field names, not on the types those fields hold, so two records that happen to declare the exact same field names cannot be told apart by it. This is not a hypothetical case, `TwoPSet`, `OpTwoPSet`, and `UWMap`'s own internal type `t` all declare a record with exactly the fields added and removed, differing only in what those fields actually contain. Because nothing in the specification's correctness is meant to depend on the order in which a hash table happens to be traversed, this is best understood as a possible source of fragility in the mechanism, rather than a guaranteed property, one that has not surfaced as a problem for the ten CRDTs implemented so far, but that a more principled version of this check, comparing field types as well as field names, would remove entirely. The sole exception to name-based matching is the single-field literal `{ payload : e }`, used by `create` whenever a module's payload is not itself a record, as discussed in Section 4.1.

4.5.3 Resolving Types for Collection Calls

`set` and `map` operations are shared across every CRDT in the specification, but the concrete element, key, and value types are different in each module, an abstract element type in one CRDT, and a pair of tagged values in another. A call such as `set.add(v, s)` or `map.get(k, m)` must therefore be typed according to where it occurs, and the compiler resolves this for each call individually, rather than assuming one type shared by every use of `set` or `map` in the specification.

In cases where an argument already carries the type required, the compiler reads it directly from that argument. `set.add(v, s)` infers the element type from `v` itself, and then verifies that `s` is a set of that same element type. `map.get(k, m)` infers the key and value types from `m`, and then checks `k` against the key type thereby obtained. Some calls have no such argument to read from, `set.empty()` takes no arguments, and `map.const(default)` supplies only a value, never a key. For these cases, the compiler falls back on the type expected of the whole expression by its surrounding context, the same expected type threaded throughout the rest of the type checker, for instance the declared type of the record field the call provides the value for. If no expected type is available at that point, the compiler reports an error rather than attempting to guess.

`map.set(k, v, m)` illustrates why this inference is handled per operation rather than through a single, uniform rule. It first attempts to read the key and value types from `m`, as `map.get` does, and only falls back to inferring them from `k` and `v`, checking `m` against the resulting map type, if that first attempt fails. Either direction is sufficient to type the call correctly, and attempting the more informative one first avoids depending on context that is not always available. This per-operation handling is what lets `set` and `map` behave as a single, shared vocabulary across every CRDT in the specification, rather than requiring a distinct set of operations for every element, key, or value type a given module requires.

COMPILING TO VERIFx AND WHY3

Chapter 4 described the language a programmer writes in, and the constraints the compiler checks before accepting a specification as valid. This chapter turns to what happens next, how a single, validated specification, one abstract syntax tree, becomes two independent artefacts, a VeriFx implementation ready for automated proof, and a Why3 formalisation ready for deductive proof, without the programmer ever writing either one directly.

Four sections follow. Section 5.1 covers the Why3 backend, the auxiliary and main module pattern CvRDTs compile through, how the five built-in axiom properties become real Why3 axioms rather than text repeated per CRDT, and the structural rules that decide how a declaration is printed once its body is known. Section 5.2 covers the same ground for VeriFx, the class and companion object pattern, the fixed proof template behind each axiom property, and the two patterns the compiler recognises to translate a quantifier into a real collection method. Section 5.3 steps back from either backend individually and looks at how the compiler chooses between them, the shape of a module’s payload, decides which of several generators handles it, and the ten CRDTs this dissertation evaluates are used throughout as concrete instances of each shape. Section 5.4 closes the chapter with RGA, the one specification where every mechanism introduced earlier applies within a single module at once.

5.1 Why3 Backend

5.1.1 Auxiliary and Main Modules

CvRDTs compile to Why3 as a pair of modules, an auxiliary module and a main module, following the same shape. CmRDTs, described through a single `execute` operation rather than a `merge` and a `compare`, do not follow this pattern, a distinction Section 5.3 comes back to. `GCounter_CRDT` is the clearest example of the CvRDT case, and the one this section builds around.

```
module GCAuxiliary
  use int.Int, int.MinMax
  type payload = int
```

```

type t = { payload: payload; }

let function get_payload (a: t) : payload = a.payload
let ghost function create () : t = { payload = 0 }
let function merge (a b: t) : t = { payload = max a.payload b.payload }
predicate compare (a b: t) = a.payload = b.payload
let function increment (v: int) (a: t) : t
  requires { v > 0 }
  = { payload = a.payload + v }
end

module GCounter_CRDT : CvRDT
  use int.Int, int.MinMax
  use GCAuxiliary as GCAux
  type payload = GCAux.payload
  type t = GCAux.t

  let function get_payload (a: t) : int = GCAux.get_payload a
  let ghost function create () : t = GCAux.create ()
  let function merge (a b: t) : t = GCAux.merge a b
  predicate compare (a b: t) = GCAux.compare a b
  predicate equals (a b: t) = compare a b /\ compare b a
  let function increment (v: int) (a: t) : t
    requires { v > 0 }
    = GCAux.increment v a
end

```

Listing 5.1: GCAuxiliary and GCounter_CRDT, as generated for Why3

The auxiliary module, `GCAuxiliary`, defines the state representation concretely, `type t = { payload: payload; }`, and every operation on it, `merge`, `compare`, and `increment` among them, as an ordinary function or predicate, with nothing hidden or restricted. The main module, `GCounter_CRDT`, exposes the same operations, but does so by declaring `type t = GCAux.t` and delegating each definition straight to `GCAux`, all while being declared against the `CvRDT` interface with `module GCounter_CRDT : CvRDT`.

This is not just naming a contract. It changes what the module can do. Once a module is written `: CvRDT`, Why3 treats its `type t` as invisible to code outside it, exactly what is needed to reason abstractly about the CRDT satisfying its interface, but also exactly what makes the module unusable as a building block for another. `PNCounter_CRDT`, composed out of two `GCounter_CRDT` instances, Appendix E, Listing E.2, needs to read and construct values of the underlying counter’s own state directly, its own `type t` carries an `incs` and a `decs` field of that type, and its `increment` calls the underlying counter’s `increment` on one of them directly. None of this is possible through the hidden `GCounter_CRDT` module. It is only possible through `GCAuxiliary`, which is exactly what `PNCounter_CRDT`’s generated code imports, `use GCAuxiliary as GCAux`, never `GCounter_CRDT` itself. The split exists to

get both properties at once, a module Why3 can hold to the CvRDT contract, and a module visible enough for other CRDTs to build on. Composed CRDTs like PNCCounter_CRDT do not repeat this split for themselves, they compile to a single module, reusing their components' auxiliary modules directly rather than generating their own, a distinction Section 5.3 returns to.

5.1.2 Axioms as Proof Obligations

The five axiom properties a specification can declare, Section 4.2.2, do not turn into text the compiler writes out per CRDT. CvRDT and CmRDT declare them once, directly as Why3 axioms, quantified over their own abstract type `t`:

```
module CvRDT
...
axiom merge_commutative: forall a b: t.
  equals (merge a b) (merge b a)
axiom merge_idempotent: forall a: t.
  equals (merge a a) a
axiom merge_associative: forall a, b, c: t.
  equals (merge (merge a b) c) (merge a (merge b c))
axiom compare_correct: forall a, b: t.
  equals a b <-> a = b
end

module CmRDT
...
axiom op_commutative: forall o1 o2: operation, a: t.
  equals (execute o2 (execute o1 a)) (execute o1 (execute o2 a))
end
```

Listing 5.2: The built-in axioms, declared inside CvRDT and CmRDT

Because these axioms live inside CvRDT/CmRDT, any module declared against one of them inherits the obligation to prove the properties for its own merge or execute, without the compiler writing a single axiom line inside GCounter_CRDT, or any other CRDT module, at all: in Why3, no CRDT module declares its own copy of these four. VeriF_x follows a parallel strategy for the same reason, the base convergence proof lives once in CvRDTProof, and each CvRDT triggers it with a single line, object GCounter_CRDT extends CvRDTProof[GCounter_CRDT], rather than restating it.

The situation is different for an axiom attached directly to a value inside a module, rather than to the interface. `merge_values` and `compare_values` in NestedMap are not part of CvRDT, so there is no shared interface for their axioms to live in instead, they compile straight into NMAuxiliary itself, one real axiom per property, quantified over the abstract value type `'v`:

```

val function merge_values (v1 v2: 'v) : 'v

axiom merge_values_commutative: forall v1 v2: 'v.
  merge_values v1 v2 = merge_values v2 v1
axiom merge_values_idempotent: forall v: 'v.
  merge_values v v = v
axiom merge_values_associative: forall v1 v2 v3: 'v.
  merge_values (merge_values v1 v2) v3 = merge_values v1 (merge_values v2 v3)

predicate compare_values (v1 v2: 'v)

axiom compare_values_correct: forall v1 v2: 'v.
  (compare_values v1 v2 /\ compare_values v2 v1) <-> v1 = v2

```

Listing 5.3: merge_values and its axioms, generated inside NMAuxiliary

The naming pattern is the same one seen above, `<function>_<property>` for three of the four, `<function>_correct` for equivalence, this time the function name is part of the generated axiom name too, since a module could have more than one such axiom.

5.1.3 Printing Types and Expressions

Two structural rules decide how a declaration is printed once its body is known. The first is about return type: any function returning boolean compiles to a Why3 predicate, never to a function, regardless of anything else about it. This is why `compare` and `equals` appear as predicate in Listing 5.1 while `merge` and `increment` appear as function, `compare` and `equals` return boolean, the other two do not.

The second concerns `create` specifically, and is a fixed exception rather than a rule: every `create`, in every CRDT, compiles to `let ghost function`, independently of what its own body does. `create` exists to give proofs an initial state to reason from, no executable program ever needs to call it, so the compiler always marks it `ghost`, stripping it out of any extracted executable code, while every other function is judged on its own body, becoming an ordinary `let function` unless it happens to call one of a fixed set of Why3 set or map operations that are themselves `ghost-only`, `Fset.union` and `Map.set` among them, in which case it becomes `ghost` too, since nothing built out of a `ghost-only` piece can be executable itself.

5.2 VeriFx Backend

5.2.1 Classes and the Proof Trait

Every module compiles to VeriFx as a class, plus a companion object of the same name. `GCounter_CRDT` shows the pattern at its simplest:

```

import org.verifx.practical.crdts.CvRDT
import org.verifx.practical.crdts.CvRDTProof

```

```

class GCounter_CRDT(payload: Int) extends CvRDT[GCounter_CRDT] {

  def value(): Int = {
    this.payload
  }

  def increment(v: Int) = {
    if ((v > 0)) {
      new GCounter_CRDT(this.payload + v)
    } else {
      this
    }
  }

  private def max(a: Int, b: Int): Int = {
    if (a >= b) a else b
  }

  def merge(that: GCounter_CRDT) = {
    new GCounter_CRDT(this.max(this.payload, that.payload))
  }

  def compare(that: GCounter_CRDT): Boolean = {
    this.payload == that.payload
  }
}

object GCounter_CRDT extends CvRDTProof[GCounter_CRDT]

```

Listing 5.4: GCounter_CRDT, as generated for VeriFx

The class extends `CvRDT[GCounter_CRDT]`, the module's own name passed back as the type parameter, which is what lets the trait's default methods (`equals`, among others) refer to `this.asInstanceOf[T]` and get `GCounter_CRDT` back. The companion object extends `CvRDTProof[GCounter_CRDT]`, the single line that triggers the axiom-derived proofs covered in the next subsection, present whenever the module's interface carries `@proof`.

`CmRDTs` follow the same overall shape, but reach it differently, since the specification only ever defines a single `execute`, never a `prepare` or an `effect` directly. `OpCounter_CRDT` shows this transformation completely:

```

class OpCounter_CRDT(ctr: Int) extends CmRDT[operation, operation,
                                     OpCounter_CRDT] {
  def inc() = new OpCounter_CRDT(this.ctr + 1)
  def dec() = new OpCounter_CRDT(this.ctr - 1)

  def prepare(op: operation) = op

  def effect(op: operation) = op match {
    case Inc() => this.inc()
    case Dec() => this.dec()
  }
}

object OpCounter_CRDT extends CmRDTProof[operation, operation, OpCounter_CRDT]

```

Listing 5.5: OpCounter_CRDT, showing the execute to prepare/effect split

prepare becomes the identity here, since the operation itself already carries everything effect needs, nothing about the replica’s own state has to be folded in before sending it. effect becomes the match on execute’s own body, with each case turned into a call to a small helper method, inc/dec, extracted from what that case computed. Why3 needs neither of these, it uses execute directly, exactly as written, with no equivalent split.

5.2.2 Axioms as Proof Templates

Each of the five properties behind axiom <property>(<function>) has a single, fixed VeriF_x proof shape behind it, the same shape every time, with only the function’s own name changing from one CRDT to the next. CvRDTProof, generated once and shared by every CvRDT that declares [@proof], shows all four:

```

trait CvRDTProof[T <: CvRDT[T]] {

  proof is_a_CvRDT {
    forall(x: T, y: T, z: T) {
      ( x.reachable() && y.reachable() && z.reachable() &&
        x.compatible(y) && x.compatible(z) && y.compatible(z) ) =>: {
        x.merge(x).equals(x) &&
        x.merge(y).equals(y.merge(x)) &&
        x.merge(y).merge(z).equals(x.merge(y.merge(z))) &&
        x.merge(y).reachable() &&
        x.merge(y).merge(z).reachable() &&
        x.compatible(y) == y.compatible(x)
      }
    }
  }

  proof mergeCommutative {
    forall (x: T, y: T) {

```

```

    (x.reachable() && y.reachable() && x.compatible(y)) =>: {
      x.merge(y).equals(y.merge(x)) &&
      x.merge(y).reachable()
    }
  }
}

proof mergeIdempotent {
  forall (x: T) {
    x.reachable() =>: x.merge(x).equals(x)
  }
}

proof mergeAssociative {
  forall (x: T, y: T, z: T) {
    ( x.reachable() && y.reachable() && z.reachable() &&
      x.compatible(y) && x.compatible(z) && y.compatible(z) ) =>: {
      x.merge(y).merge(z).equals(x.merge(y.merge(z))) &&
      x.merge(y).merge(z).reachable()
    }
  }
}

proof compareEquivalence {
  forall (x: T, y: T) {
    x.equals(y) == (x == y)
  }
}
}

```

Listing 5.6: CvRDTProof, generated once and shared by every CvRDT

`is_a_CvRDT` itself is worth a second look. It states, in one proof, most of what `mergeCommutative`, `mergeIdempotent`, and `mergeAssociative` state separately. The two are not redundant so much as complementary, `is_a_CvRDT` is what actually proves the module is a valid join-semilattice as a whole, while the three named proofs let a failure be identified with one specific property rather than a single combined obligation, which is why the compiler generates both rather than one or the other.

5.2.3 Translating Quantifiers

A `forall` or `exists` written over a collection compiles to a real VeriF_x collection method, `.forall`, `.exists`, never to a literal quantifier, which VeriF_x only accepts inside a proof block. The compiler recognises one specific shape for this, a variable tested for membership in a set, combined with a condition on that same variable, and rewrites it directly. RGA's `isNewTimestamp` shows this working on a single variable. In the specification, this is written as:

```
forall(vertex: elem) { !set.contains(vertex, a.added) ||
    vertex.timestamp != t }
```

and compiles to:

```
this.vertices.added.forall((vertex: Elem[V]) => (vertex.timestamp != t))
```

A second, more specific shape is recognised for comparing two maps, key by key, rather than testing membership in a single set. `UWMap`'s `compare` is written this way in the specification, once for its `added` field and once for its `removed` field, and compiles to:

```
def compare(that: UWMap[V]): Boolean = {
  this.added.keys().forall((key: Int) =>
    this.added.get(key).subsetOf(that.added.getOrElse(
      key, new Set[Tuple[V, Int]]())))) &&
  this.removed.keys().forall((key: Int) =>
    this.removed.get(key).subsetOf(
      that.removed.getOrElse(key, new Set[Int]()))))
}
```

each half iterating one side's own keys and using `getOrElse` to fill in a default for the other side wherever a key is missing from it, rather than requiring both maps to share exactly the same keys, the rule applied twice here, once per field.

Neither rule is a general quantifier translator, each recognises one specific shape and rewrites exactly that shape. A `forall` or `exists` that matches neither is rejected.

Quantifiers over a vector, rather than a set or a map, follow yet another path, specific to the vector-indexed shape and covered together with it in the next section.

5.3 Compiling Different CRDTs

5.3.1 Generation Rules

Sections 5.1 and 5.2 each described one generic pattern per backend, the auxiliary and main module pair for `Why3`, the class and companion object for `VeriFx`, built around `GCounter_CRDT` as the simplest possible CRDT. Neither backend, in practice, treats every CRDT this way. Both inspect the shape of a module's payload, or its explicit type `t`, where one exists, before choosing how to generate it, though the two backends do not always make the same distinctions.

A payload that is just an integer or boolean needs nothing beyond the ordinary generator on either side, the one already described for `GCounter_CRDT`, and the same generator handles a plain record just as well, which is how `PNCounter_CRDT` and `TwoPSet` are both built. `GSet`, whose payload is exactly `set elem`, triggers a dedicated set generator on both sides instead, the only one of the ten built this way. `NestedMap` is parametrised over a

key type and a value type at once, which is what triggers VeriFx’s map-poly generator, the only one of the ten to need it. Declaring an explicit type `t`, rather than letting the compiler derive one from `payload`, triggers VeriFx’s dedicated generator for that case before anything else about the module is considered, as it does for both `VGCounter` and `UWMap`. The `CmRDT` interface follows the same logic but reaches a different outcome, since the interface itself is different, `OpCounter_CRDT`’s payload is just an integer, and compiles through the same kind of generator a payload like that would use on the `CvRDT` side, but Why3 gives it no auxiliary module, since the two-module split only exists for an interface declaring both `merge` and `compare`, and `CmRDT` declares `execute` instead. `OpTwoPSet` and `RGA`, whose payload is a record, compile through the corresponding `CmRDT` record generator.

5.3.2 Sets and Generic Maps

`GSet` is the one CRDT with a payload of exactly `set elem`. Both backends detect this precisely and route it to a generator built only around set operations, `add`, `union`, `subset`, translated directly to their native VeriFx equivalents and to Why3’s `Fset` module, without the general record handling the other generators need.

`NestedMap` is the only CRDT parametrised over two abstract types at once, a key type and a value type, type `payload = map<k, v>`. On the Why3 side this needs nothing special, Why3’s own polymorphism represents `k` and `v` as type variables directly, and the generic auxiliary and main module pair handles it exactly as it would a concrete type. VeriFx has no equivalent native polymorphism over two independent collection type parameters in the same way, so it uses a dedicated generator that builds the class with both type parameters declared up front, `class NestedMap[K, V]`, and carries them through every method.

5.3.3 Composing CRDTs

`PNCounter_CRDT` (`{ incs, decs, payload }`) and `TwoPSet` (`{ added, removed }`) both have a payload that is a record, and neither is treated differently on the VeriFx side, both compile through the same generic class generator as `GCounter_CRDT`, which already knows how to turn a record type into a constructor’s parameter list. Why3 treats the two differently from each other, though, `TwoPSet`’s two set fields are local to it, so it goes through the generic auxiliary and main module pair, while `PNCounter_CRDT`’s fields are typed `GCounter_CRDT.payload`, referencing another module’s type directly. The composite generator exists to detect exactly this and reuse what already exists rather than duplicate it, `PNCounter_CRDT`’s own `merge` calls `GCounter_CRDT`’s, through `GCAuxiliary`, instead of a second, independent implementation being generated for it.

`OpTwoPSet` is the first `CmRDT` whose payload is a record. A `CvRDT` with a record payload still goes through the same generic generator regardless of what the record contains, but a `CmRDT` does not, an integer payload and a record payload are handled by two separate generators. `RGA`’s payload is also a record, and is compiled by the same generator as

OpTwoPSet's for that reason. The next section returns to this generator, since most of what makes compiling RGA distinctive, the `[@vfx class: ...]` extraction, the `[@vfx Module: field]` reuse, the fuel elimination, happens inside it.

5.3.4 Vector and Explicit State

VGCounter and UWMap are the only two CRDTs that declare their own type `t` directly, rather than letting the compiler derive one from payload, and on the VeriFx side this is the very first thing checked, before a payload's structure is even considered. Both compile through VeriFx's dedicated generator for an explicit type `t`, which is also where the two directives from Subsection 4.4.2 actually take effect, the field type override behind `[@vfx Vector[Int]]`, and the class reuse behind `[@vfx Module: field]`, nothing else in the compiler reads those attributes at all. Why3 needs none of this distinction, an explicit type `t` is still just a record as far as the generic auxiliary and main module pair is concerned, and it is generated exactly as GCounter_CRDT's is, the only difference visible in the generated code being that the record was written by the programmer instead of derived by the compiler.

5.4 Compiling RGA

Every mechanism this chapter has introduced separately, class extraction, field composition, fuel elimination, quantifier translation, is triggered by something written in the specification itself, a `[@vfx class: ...]` directive, a fuel parameter, a quantifier over a set. RGA happens to be the one specification, among the ten, that uses all of them at once. This section brings the four together, and looks in more depth at the one not yet covered on its own, the composition of added and removed into vertices, triggered the same way as the others, by a directive written on the payload declaration itself.

RGA's payload declares added and removed as two ordinary set `elem` fields, the same shape OpTwoPSet's own payload has. The `[@vfx OpTwoPSet: vertices]` directive on that declaration replaces both fields with a single one, typed as an instance of OpTwoPSet itself, in the generated class:

```
class RGA[V] (clock: LamportClock, vertices: OpTwoPSet[Elem[V]],
             edges: Map[Elem[V], Elem[V]], bottom_value: V)
  extends CmrDT[RGASOp[V], RGAMsg[V], RGA[V]]
```

Every method that used to read or rebuild added and removed together is rewritten to go through vertices instead, though not always the same way. Where OpTwoPSet already has a method for what is needed, the generated code simply calls it, lookup becomes `this.vertices.lookup(v)`, delegating to a method OpTwoPSet has already had its own convergence proved for, rather than restating `set.contains(v, added) && !set.contains(v, removed)` locally. Where no such method exists, `preAddRightDownstream` needs to

check membership in `added` alone, without the accompanying check on `removed` that `lookup` always performs, the generated code reaches directly into the composed object's own field instead, `this.vertices.added.contains(1)`, since `OpTwoPSet` does not expose a method for only that check on its own.

Constructing a new state shows the same reuse from the other direction. The specification builds a new payload by writing out `added` and `removed` directly, as seen already in `find_insert_position`, but the generated code replaces that with a single call, `this.vertices.add(e)`, since `OpTwoPSet`'s own `add` already adds to one set while carrying the other through unchanged, exactly what the specification asked for. `removeDownstream` does the same with `this.vertices.rem(w)`. In both cases, what the specification writes as a two-field record update compiles to one method call on an already-verified component, rather than two fields being managed separately, one at a time.

None of this is visible on the `Why3` side. `RGA`'s `Why3` payload declares `added` and `removed` as plain `fset elem` fields, local to `RGA` itself, exactly as `OpTwoPSet`'s own `Why3` module declares them, with no reference between the two, and `limport_clock` stays an ordinary record type inside `RGA`'s own module rather than being extracted anywhere. Both `VeriFx` directives, the class extraction and the field composition, exist only for `VeriFx`, and have no effect when the same specification is compiled for `Why3`.

CASE STUDIES AND EVALUATION

Chapters 4 and 5 described the specification language and the compiler built on top of it, from the inside, what the language lets a programmer write, and how the compiler turns that into VeriFx and Why3 code. This chapter turns to what comes next, what happens once that generated code actually reaches a solver, across the ten CRDTs this dissertation specifies.

Two sections follow. Section 6.1 compares lines written in the specification language against lines generated for each backend, across all ten CRDTs, and looks at why the two backends do not grow from the same specification at the same rate, or even in the same direction. Section 6.2 turns to the proofs themselves, nine of the ten CRDTs are proved in both VeriFx and Why3 without incident, PNCOUNTER_CRDT needs one small manual step in Why3 to close a proof neither solver could reach directly, and RGA is the one CRDT where the two backends genuinely disagree, proved in Why3, rejected entirely in VeriFx, for reasons the chapter traces back to exactly the mechanisms Chapters 4 and 5 already introduced.

6.1 Specification Effort

Chapters 4 and 5 demonstrated, simply by how the language is built, that the framework reduces how much a programmer has to write to target both VeriFx and Why3 at once. This section puts a number on that argument, across the ten CRDTs evaluated throughout this dissertation, comparing lines written in the specification language against lines generated for each backend.

The comparison also serves a second purpose beyond quantifying effort. Having the same specification available in both languages at once means a programmer is not locked into whichever tool happens to handle a given CRDT. RGA is the clearest case of this in the evaluation, its Why3 proof succeeds where its VeriFx proof does not, and it is only because both are generated from the same specification that the two can be compared at all.

The LoC metric excludes the framework’s global interface and directive definitions, counting only the code generated specific to each CRDT.

CRDT	Model	Written LoC	LoC (VeriFx)	LoC (WhyML)
G_Counter	State	13	24	35
PN_Counter	State	18	23	25
VG_Counter	State	26	27	51
Op_Counter	Operation	14	17	19
NestedMap	State	29	28	57
GSet	State	15	9	37
TwoPSet	State	17	16	43
TwoPSet	Operation	28	25	22
UWMap	State	45	20	58
RGA	Operation	165	170	81

Table 6.1: Manual specification effort versus code generated, by CRDT.

Across the ten CRDTs, the specification does not translate into more code on both sides equally, one backend tends to expand it, the other often does not. Why3 output is larger than the specification for every CRDT except `OpTwoPSet` and `RGA`, one cause of this is the auxiliary and main module split, where in every `CvRDT`, every function is written out twice, a full definition first, then a second version that only calls it. This duplication is deliberate, not incidental. Section 5.1.1 showed why, declaring a `CvRDT`’s main module against the `CvRDT` interface makes Why3 treat its type `t` as abstract, exactly what is needed to reason about the module satisfying the interface, but that same abstraction makes the module unusable as a building block for another CRDT, `PNCounter_CRDT` needs to read and construct `GCounter_CRDT`’s state directly, something the hidden main module cannot expose. The auxiliary module exists to give composed CRDTs that access, so every function written out twice is the price paid for two properties Why3 does not otherwise let a single module have at once, an interface abstract enough to prove against, and a concrete enough implementation for other CRDTs to build on. VeriFx output, on the other hand, is smaller than the specification for half of the ten, most dramatically for `GSet`, which writes 15 lines but generates only 9, and nearly as much for `UWMap`, which writes 45 but generates only 20. This follows directly from how the two backends are built, unlike Why3, VeriFx implements a function directly, without a second module declaring its signature first.

The pattern also splits cleanly along the two interfaces. Among the seven `CvRDTs`, Why3’s output is the largest of the three columns consistently, the auxiliary and main module duplication persists regardless of how simple or complex the CRDT itself is. Among the three `CmRDTs`, no such rule holds, `OpCounter_CRDT`, the simplest of the three, still generates slightly more Why3 than VeriFx, but the relationship reverses as the `CmRDT` grows more complex, `OpTwoPSet`’s VeriFx output overtakes its Why3 output by a small margin, and `RGA`’s VeriFx output is more than twice its Why3 output. Both `OpTwoPSet` and `RGA` declare several `[@vfx]` functions, `preRemove`, `enabledSrc`, `reachable`, `isNewTimestamp` among `RGA`’s, that exist purely to support VeriFx’s own proof machinery and are entirely absent from the Why3 output. The reason is how VeriFx scopes its `is_a_CmRDT` obligation, Section 4.4.1 already noted that `enabledSrc`, `enabledDown`, `compatibleS`, and `reachable`

are methods every `CmRDT` inherits a default implementation for, and that default returns `true` unconditionally, so the proof by default considers every state reachable and every operation admissible. Why3 needs none of this overhead, its `op_commutative` axiom is stated once, unconditionally, and proved, where a proof is needed, by reasoning directly about `execute`'s own body, rather than first having to characterise, in a separate function, which states and operations the proof is even allowed to consider. A `CmRDT` with more of these functions has its VeriF_x output grow specifically, while Why3 never sees them, which is what causes the reversal as complexity increases.

The more informative comparison here is not the line count, but the structure of what gets generated, the same specification produces an auxiliary and main module pair on one side and a set of native collection method calls on the other, each idiomatic to its own tool rather than a direct translation of the other.

6.2 RGA Proof

Every CRDT specified in this dissertation is proved in Why3, and nine of the ten need no human input beyond selecting a prover, Z3 or Alt-Ergo discharge every proof obligation on its own. The one exception is `PNCOUNTER_CRDT`'s `compare_correct`, whose statement, `equals a b <-> a = b`, neither prover could close directly. The fix needed only one step, writing `assert (t'eq a b)` in the Why3 IDE to isolate the equality `a = b` as its own intermediate goal, rather than leaving it embedded inside the full biconditional. Once isolated this way, both halves, proving the assertion, and then using it to close the original goal, are closed automatically in a fraction of a second each, a fact the automatic search had already failed to reach once, inside the larger, combined goal. This is a common pattern for the manual effort deductive verification asks for, not new information supplied by the programmer, but a smaller, more tractable search than the one the solver was originally given.

The outcome is different for VeriF_x. Nine of the same ten CRDTs are proved automatically by Z3 there too, but the tenth is not `PNCOUNTER_CRDT`, it is `RGA`, and no manual step closes the gap the way this assertion did for Why3, the tool reports `Could not prove (RGA, is_a_CmRDT) [rejected]` outright. This is not an issue this dissertation alone has observed, VeriF_x's own paper reports the same CRDT as the one exception in an otherwise large collection it verifies successfully, timing out specifically on `RGA` because of the recursive nature of its insertion algorithm [19]. `RGA` is therefore the only one of the ten CRDTs where the two backends genuinely disagree on the outcome.

The proof that fails on the VeriF_x side, `is_a_CmRDT`, depends on reasoning about `find_insert_position` across an arbitrary number of recursive calls, one per existing vertex the new insertion has to walk past. On the Why3 side, this reasoning is not left to the solver to discover, it is handed to it directly, in the form of a lemma proved by induction on `fuel`:

```

lemma find_insert_position_facts (stamp: lamport_clock, e left right: elem,
                                a: payload, fuel: integer)

  variant (fuel)
  ensures { (find_insert_position(stamp, e, left, right, a, fuel)).added ==
            set.add(e, a.added) }
  ensures { (find_insert_position(stamp, e, left, right, a, fuel)).removed ==
            a.removed }
= if fuel <= 0 then true
  else if smaller_clock(stamp, right.timestamp) then
    find_insert_position_facts(stamp, e, right, successor(right, a), a, fuel - 1)
  else true

```

Listing 6.1: An inductive proof of `find_insert_position`, by `fuel`

The base case, `fuel <= 0`, is immediate: it matches exactly the base case `find_insert_position` itself takes when `fuel` runs out, inserting `e` directly and leaving `removed` untouched, precisely what the two `ensures` clauses state. The third branch, where `find_insert_position` also inserts `e` directly rather than recursing, needs no separate case here either, it is covered by the same reasoning as the base case, which is why the lemma’s own body folds it into the same `true`. Only the middle branch, where `find_insert_position` recurses, needs the lemma to recurse too, and that recursive call is itself the induction hypothesis, already established for a smaller `fuel` by the time it is needed. Why3 folds this whole argument, induction hypothesis included, into a single verification condition, which Z3 closes automatically in one call, rather than needing to reason about the unbounded recursion directly.

VeriFx’s `find_insert_position` has no `fuel` parameter at all. `fuel` exists only so Why3 has a decreasing quantity to prove termination against, a requirement specific to deductive verification that VeriFx does not share, so the compiler removes it during compilation, along with the base case that only existed to give Why3 something to terminate on. What VeriFx receives instead is an ordinary, unbounded recursive function, with no companion lemma, no induction structure, nothing beyond the function itself, for Z3 to reason from. This is exactly the limitation the Abstract and Section 3.3.3 both raise, an SMT solver reasoning about a recursive definition with no bound on how many times it may call itself has no general way to conclude a property holds for every possible depth, only for however many unfoldings it happens to try before giving up. RGA is the one CRDT among the ten whose correctness depends on this kind of unbounded reasoning, which is exactly why it is also the one CRDT where the difference between automated and deductive verification stops being theoretical and starts being the difference between a proof and a rejection.

CONCLUSION AND FUTURE WORK

7.1 Conclusion

Conflict-Free Replicated Data Types promise convergence without coordination, but the two established ways of verifying that promise make different trade-offs. Automated tools built on SMT solvers, VeriFx among them, prove algebraic properties quickly and with little manual effort, but struggle precisely where a CRDT’s correctness depends on recursive definitions or unbounded structures. Deductive platforms such as Why3 handle exactly this kind of reasoning, but ask the programmer to write and maintain the proof by hand, in a language with its own syntax and its own demands. Neither approach, used alone, covers what the other is good at.

This dissertation addressed that gap with a single specification language, compiled automatically to both VeriFx and Why3 from the same source. The language hides the syntactic differences between the two targets behind one set of constructs, interfaces, axioms, and a handful of directives that let a specification diverge between backends where it needs to, and the compiler derives an idiomatic implementation for each, an auxiliary and main module pair or a set-based predicate for Why3, a class extending a shared proof trait for VeriFx, from the same abstract syntax tree.

The evaluation, across ten CRDTs of increasing complexity, showed this design working as intended. Specification effort dropped for half of the ten, most sharply for the CRDTs built around sets, where a single native collection method absorbs logic the specification states explicitly. Every one of the ten is proved automatically in Why3, Z3 or Alt-Ergo closing every obligation on their own except one, `PNCounter_CRDT`’s `compare_correct`, where a single manual assertion was enough to split a goal that the solver could not reach on its own into two the solver closed immediately. VeriFx proves nine of the same ten CRDTs just as automatically, but rejects the tenth, `RGA`, the one CRDT whose correctness genuinely depends on reasoning about an unbounded recursive structure, exactly the case the literature already identifies as beyond the reach of SMT solving alone. Because the same specification also compiles to Why3, `RGA` still gets proved, an inductive lemma over the same `fuel` parameter the compiler strips out for VeriFx is enough for Why3 to

close the proof VeriFx cannot. RGA is, in that sense, the clearest demonstration of why this dissertation paired the two approaches instead of choosing between them.

7.2 Future Work

RGA is the only one of the ten CRDTs whose specification needed the `fuel` and `variant` pattern to give Why3 a termination measure to induct on, and the only one whose correctness depends on reasoning about a recursive structure of unbounded size in the first place. Extending the evaluation to further recursive or unbounded CRDTs, sequence CRDTs other than RGA among them, would test whether this same pattern carries over with little change, or whether it surfaces cases where the language or the compiler would need to grow to support them.

A related idea follows from the same divergence, the two backends currently prove every property entirely independently, even when one has already succeeded exactly where the other has not. RGA illustrates this directly, Why3 proves RGA's convergence through the `find_insert_position_facts` lemma, while VeriFx is asked to establish the same property on its own, as `is_a_CmRDT`, and rejects it. Rather than treating the two proofs as fully separate, the compiler could let a property validated in one backend become an assumption the other is allowed to rely on, an axiom generated from Why3's proof, letting VeriFx's own `is_a_CmRDT` obligation lean on it instead of reasoning about the same unbounded recursion on its own, and symmetrically for a property only VeriFx manages to close automatically. This would only be sound if the translation between the two representations preserves exactly the property being trusted, a stronger guarantee than compiling both from the same specification currently provides on its own.

BIBLIOGRAPHY

- [1] C. W. Barrett, C. L. Conway, M. Deters, L. Hadarean, D. Jovanovic, T. King, A. Reynolds, and C. Tinelli. “CVC4”. In: *Computer Aided Verification - 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14-20, 2011. Proceedings*. Ed. by G. Gopalakrishnan and S. Qadeer. Vol. 6806. Lecture Notes in Computer Science. Springer, 2011, pp. 171–177. doi: [10.1007/978-3-642-22110-1_14](https://doi.org/10.1007/978-3-642-22110-1_14) (cit. on pp. 14, 19).
- [2] J. Bauwens and E. G. Boix. “Nested Pure Operation-Based CRDTs”. In: *37th European Conference on Object-Oriented Programming, ECOOP 2023, Seattle, Washington, United States, July 17-21, 2023*. Ed. by K. Ali and G. Salvaneschi. Vol. 263. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 2:1–2:26. doi: [10.4230/LIPICS.ECOOP.2023.2](https://doi.org/10.4230/LIPICS.ECOOP.2023.2) (cit. on pp. 9, 11, 16).
- [3] Y. Bertot and P. Castéran. *Interactive Theorem Proving and Program Development - Coq’Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004. ISBN: 978-3-642-05880-6. doi: [10.1007/978-3-662-07964-5](https://doi.org/10.1007/978-3-662-07964-5) (cit. on p. 19).
- [4] D. Borrego, N. M. Preguiça, E. G. Boix, and C. Ferreira. “Ensuring Convergence and Invariants Without Coordination”. In: *39th European Conference on Object-Oriented Programming, ECOOP 2025, Bergen, Norway, June 30 - July 2, 2025*. Ed. by J. Aldrich and A. Silva. Vol. 333. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025, 4:1–4:29. doi: [10.4230/LIPICS.ECOOP.2025.4](https://doi.org/10.4230/LIPICS.ECOOP.2025.4) (cit. on pp. 5, 11).
- [5] S. Conchon, A. Coquereau, M. Iguernlala, and A. Mebsout. “Alt-Ergo 2.2”. In: *SMT Workshop: International Workshop on Satisfiability Modulo Theories*. 2018 (cit. on pp. 14, 19).
- [6] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels. “Dynamo: amazon’s highly available key-value store”. In: *Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSP 2007, Stevenson, Washington, USA, October 14-17, 2007*. Ed. by

- T. C. Bressoud and M. F. Kaashoek. ACM, 2007, pp. 205–220. DOI: [10.1145/1294261.1294281](https://doi.org/10.1145/1294261.1294281) (cit. on p. 8).
- [7] J. Filliâtre and A. Paskevich. “Why3 - Where Programs Meet Provers”. In: *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*. Ed. by M. Felleisen and P. Gardner. Vol. 7792. Lecture Notes in Computer Science. Springer, 2013, pp. 125–128. DOI: [10.1007/978-3-642-37036-6_8](https://doi.org/10.1007/978-3-642-37036-6_8) (cit. on pp. 1, 2, 12, 14, 19).
- [8] S. Gilbert and N. A. Lynch. “Perspectives on the CAP Theorem”. In: *Computer* 45.2 (2012), pp. 30–36. DOI: [10.1109/MC.2011.389](https://doi.org/10.1109/MC.2011.389) (cit. on pp. 1, 4).
- [9] G. Kaki, S. Priya, K. C. Sivaramakrishnan, and S. Jagannathan. “Mergeable replicated data types”. In: *Proc. ACM Program. Lang.* 3.OOPSLA (2019), 154:1–154:29. DOI: [10.1145/3360580](https://doi.org/10.1145/3360580) (cit. on p. 12).
- [10] M. Kleppmann and A. R. Beresford. “A Conflict-Free Replicated JSON Datatype”. In: *IEEE Trans. Parallel Distributed Syst.* 28.10 (2017), pp. 2733–2746. DOI: [10.1109/TPDS.2017.2697382](https://doi.org/10.1109/TPDS.2017.2697382) (cit. on p. 11).
- [11] M. Köhler, G. Zakhour, P. Weisenburger, and G. Salvaneschi. “Consistent Local-First Software: Enforcing Safety and Invariants for Local-First Applications”. In: *IEEE Trans. Software Eng.* 51.1 (2025), pp. 53–65. DOI: [10.1109/TSE.2024.3477723](https://doi.org/10.1109/TSE.2024.3477723) (cit. on p. 8).
- [12] L. Lamport. “Time, clocks, and the ordering of events in a distributed system”. In: *Concurrency: the Works of Leslie Lamport*. Ed. by D. Malkhi. ACM, 2019, pp. 179–196. DOI: [10.1145/3335772.3335934](https://doi.org/10.1145/3335772.3335934) (cit. on p. 17).
- [13] W. Lloyd, M. J. Freedman, M. Kaminsky, and D. G. Andersen. “Don’t settle for eventual: scalable causal consistency for wide-area storage with COPS”. In: *Proceedings of the 23rd ACM Symposium on Operating Systems Principles 2011, SOSP 2011, Cascais, Portugal, October 23-26, 2011*. Ed. by T. Wobber and P. Druschel. ACM, 2011, pp. 401–416. DOI: [10.1145/2043556.2043593](https://doi.org/10.1145/2043556.2043593) (cit. on p. 5).
- [14] J. M. Lourenço. *novathesis: A LaTeX Template for Academic Theses and Dissertations*. Version 8.3.0. NOVA University Lisbon. 2026. DOI: [10.5281/zenodo.21938603](https://doi.org/10.5281/zenodo.21938603) (cit. on p. 75).
- [15] F. S. Meirim. “Static Verification of Cloud Applications with Why3”. MA thesis. Universidade NOVA de Lisboa (Portugal), 2019 (cit. on p. 14).
- [16] L. M. de Moura and N. S. Bjørner. “Z3: An Efficient SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings*. Ed. by C. R. Ramakrishnan and J. Rehof. Vol. 4963. Lecture Notes in Computer Science.

- Springer, 2008, pp. 337–340. doi: [10.1007/978-3-540-78800-3_24](https://doi.org/10.1007/978-3-540-78800-3_24) (cit. on pp. 13, 14, 17, 19).
- [17] D. Navalho, S. Duarte, and N. M. Preguiça. “A study of CRDTs that do computations”. In: *Proceedings of the First Workshop on Principles and Practice of Consistency for Distributed Data, PaPoC@EuroSys 2015, Bordeaux, France, April 21, 2015*. Ed. by C. Baquero and M. Serafini. ACM, 2015, 1:1–1:4. doi: [10.1145/2745947.2745948](https://doi.org/10.1145/2745947.2745948) (cit. on p. 8).
- [18] T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*. Vol. 2283. Lecture Notes in Computer Science. Springer, 2002. ISBN: 3-540-43376-7. doi: [10.1007/3-540-45949-9](https://doi.org/10.1007/3-540-45949-9) (cit. on p. 13).
- [19] K. D. Porre, C. Ferreira, and E. G. Boix. “VeriFx: Correct Replicated Data Types for the Masses”. In: *37th European Conference on Object-Oriented Programming, ECOOP 2023, Seattle, Washington, United States, July 17-21, 2023*. Ed. by K. Ali and G. Salvaneschi. Vol. 263. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 9:1–9:45. doi: [10.4230/LIPICS.ECOOP.2023.9](https://doi.org/10.4230/LIPICS.ECOOP.2023.9) (cit. on pp. 1, 11–13, 17, 48).
- [20] N. M. Preguiça. “Conflict-free Replicated Data Types: An Overview”. In: *CoRR abs/1806.10254* (2018). arXiv: [1806.10254](https://arxiv.org/abs/1806.10254) (cit. on pp. 4, 5, 7, 15).
- [21] A. d. R. M. Rijo. “Building Tunable CRDTs”. MA thesis. Universidade NOVA de Lisboa (Portugal), 2018 (cit. on p. 9).
- [22] D. dos Santos Borrego. “Verifying and Enforcing Application Constraints in Antidote SQL”. MA thesis. Universidade NOVA de Lisboa (Portugal), 2022 (cit. on p. 2).
- [23] M. Shapiro, N. M. Preguiça, C. Baquero, and M. Zawirski. “Convergent and Commutative Replicated Data Types”. In: *Bull. EATCS* 104 (2011), pp. 67–88. URL: <http://eatcs.org/beatcs/index.php/beatcs/article/view/120> (cit. on pp. 1, 5, 7–9, 12, 15, 16).
- [24] *The Why3 platform, version 1.8*. URL: <https://www.why3.org/doc/> (visited on 2026-09-21) (cit. on pp. 12, 14, 15).
- [25] E. Yanakieva, M. Youssef, A. H. Rezae, and A. Bieniusa. “Access Control Conflict Resolution in Distributed File Systems using CRDTs”. In: *PaPoC@EuroSys 2021, 8th Workshop on Principles and Practice of Consistency for Distributed Data, Online Event, United Kingdom, April 26, 2021*. ACM, 2021, 1:1–1:3. doi: [10.1145/3447865.3457970](https://doi.org/10.1145/3447865.3457970) (cit. on p. 13).

GROW-ONLY COUNTER

```
import org.verifx.practical.crdts.CvRDT
import org.verifx.practical.crdts.CvRDTProof

class GCounter_CRDT(payload: Int) extends CvRDT[GCounter_CRDT] {

  def increment(v: Int) = {
    if ((v > 0)) {
      new GCounter_CRDT(this.payload + v)
    } else {
      this
    }
  }

  def value(): Int = {
    this.payload
  }

  private def max(a: Int, b: Int): Int = {
    if (a >= b) a else b
  }

  def merge(that: GCounter_CRDT) = {
    new GCounter_CRDT(this.max(this.payload, that.payload))
  }

  def compare(that: GCounter_CRDT): Boolean = {
    this.payload == that.payload
  }

}

object GCounter_CRDT extends CvRDTProof[GCounter_CRDT]
```

POSITIVE-NEGATIVE COUNTER

```
import org.verifx.practical.crdts.CvRDT
import org.verifx.practical.crdts.CvRDTProof

class PNCCounter_CRDT(incs: Int, decs: Int) extends CvRDT[PNCCounter_CRDT] {

  def value() = {
    (this.incs - this.decs)
  }

  private def max(a: Int, b: Int) = {
    if (a >= b) a else b
  }

  def merge(that: PNCCounter_CRDT) = {
    new PNCCounter_CRDT(this.max(this.incs, that.incs), this.max(this.decs, that.decs))
  }

  def compare(that: PNCCounter_CRDT): Boolean = {
    this.incs == that.incs && this.decs == that.decs
  }

  def increment(v: Int) = {
    new PNCCounter_CRDT(this.incs + v, this.decs)
  }

  def decrement(v: Int) = {
    new PNCCounter_CRDT(this.incs, this.decs + v)
  }

}

object PNCCounter_CRDT extends CvRDTProof[PNCCounter_CRDT]
```

Two-Phase Set

```
import org.verifx.practical.crdts.CvRDT
import org.verifx.practical.crdts.CvRDTProof1

class TwoPSet[V](added: Set[V] = new Set[V](),
                 removed: Set[V] = new Set[V]()) extends CvRDT[TwoPSet[V]] {
  def lookup(element: V) =
    this.added.contains(element) && !this.removed.contains(element)
  def add(element: V) = new TwoPSet(this.added.add(element), this.removed)
  def remove(element: V) = new TwoPSet(this.added, this.removed.add(element))

  def merge(that: TwoPSet[V]) = {
    val newAdded = this.added.union(that.added)
    val newRemoved = this.removed.union(that.removed)
    new TwoPSet(newAdded, newRemoved)
  }

  def compare(that: TwoPSet[V]) = {
    this.added.subsetOf(that.added) &&
    this.removed.subsetOf(that.removed)
  }
}

object TwoPSet extends CvRDTProof1[TwoPSet]
```

WHY3 IMPLEMENTATION

```
module CvRDT

  type payload

  type t

  val ghost function create () : t

  val function get_payload (a: t) : payload

  function merge t t : t

  predicate compare t t

  predicate equals (a b: t)
  = compare a b /\ compare b a

  axiom merge_commutative: forall a b: t.
    equals (merge a b) (merge b a)

  axiom merge_idempotent: forall a: t.
    equals (merge a a) a

  axiom merge_associative: forall a, b, c: t.
    equals (merge (merge a b) c) (merge a (merge b c))

  axiom compare_correct: forall a, b: t.
    equals a b <-> a = b

end

module GCAuxiliary
```

```
use int.Int, int.MinMax

type payload = int

type t = { payload: payload; }

let function get_payload (a: t) : payload
  = a.payload

let ghost function create () : t
  = { payload = 0 }

let function merge (a b: t) : t
  = { payload = max a.payload b.payload }

predicate compare (a b: t)
  = a.payload = b.payload

let function increment (v: int) (a: t) : t
  requires { v > 0 }
  = { payload = a.payload + v }

end

module GCounter_CRDT : CvRDT

  use int.Int, int.MinMax

  use GCAuxiliary as GCAux

  type payload = GCAux.payload

  type t = GCAux.t

  let function get_payload (a: t) : int
    = GCAux.get_payload a

  let ghost function create () : t
    = GCAux.create ()

  let function merge (a b: t) : t
    = GCAux.merge a b

  predicate compare (a b: t)
    = GCAux.compare a b
```

```
predicate equals (a b: t)
= compare a b /\ compare b a

let function increment (v: int) (a: t) : t
  requires { v > 0 }
  = GCAux.increment v a

end

module PNCOUNTER_CRDT : CvRDT

  use int.Int
  use GCAuxiliary as GCAux

  type payload = int

  type t = { incs: GCAux.t; decs: GCAux.t; payload_pnc: payload }
    invariant { payload_pnc = GCAux.get_payload incs - GCAux.get_payload decs }
    by { incs = GCAux.create (); decs = GCAux.create (); payload_pnc = 0 }

  let function get_payload (a: t) : payload
    = a.payload_pnc

  let ghost function create () : t
    = { incs = GCAux.create (); decs = GCAux.create (); payload_pnc = 0 }

  let function make (a b: GCAux.t) : t
    = { incs = a; decs = b;
        payload_pnc = GCAux.get_payload a - GCAux.get_payload b }

  function increment (v: int) (a: t) : t
    = make (GCAux.increment v a.incs) a.decs

  function decrement (v: int) (a: t) : t
    = make a.incs (GCAux.increment v a.decs)

  function merge (a b: t) : t
    = make (GCAux.merge a.incs b.incs) (GCAux.merge a.decs b.decs)

  predicate compare (a b: t)
    = GCAux.compare a.incs b.incs /\ GCAux.compare a.decs b.decs

  predicate equals (a b: t)
    = compare a b /\ compare b a

end
```

CRDT SPECIFICATIONS

This appendix presents the complete specification, in the framework’s own language, for each of the ten CRDTs evaluated throughout this dissertation. Every listing here is the single source Chapter 5 compiles into both a VeriFx implementation and a Why3 formalisation, and the same source Chapter 6 measures against the code each backend generates from it. The order follows Table 6.1, from GCounter_CRDT, the simplest CvRDT in the evaluation, through RGA, the most complex of the ten and the only one where the two backends disagree.

```

module GCounter_CRDT : CvRDT
  type payload = integer

  val create : payload = { payload : 0 }

  val merge (a b: payload) : payload =
    max(a, b)

  val compare (a b: payload) : boolean =
    a == b

  val equals (a b: payload) : boolean =
    compare(a, b) && compare(b, a)

  val increment (v: integer, a: payload) : payload =
    requires { v > 0 }
    a + v
end

```

Listing E.1: GCounter_CRDT

```

module PNCOUNTER_CRDT : CvRDT
  type payload =
    { incs: GCounter_CRDT.payload,
      decs: GCounter_CRDT.payload,
      payload: integer }

```

```
invariant value(a: payload) =
  a.payload == a.incs - a.decs

val create: payload =
  { incs: GCounter_CRDT.create,
    decs: GCounter_CRDT.create,
    payload : 0 }

val make (a b: GCounter_CRDT.payload) : payload =
  { incs: a, decs: b,
    payload: GCounter_CRDT.get_payload(a) - GCounter_CRDT.get_payload(b) }

val merge (a b: payload) : payload =
  make (GCounter_CRDT.merge(a.incs, b.incs), GCounter_CRDT.merge(a.decs, b.decs))

val compare (a b: payload) : boolean =
  GCounter_CRDT.compare(a.incs, b.incs) && GCounter_CRDT.compare(a.decs, b.decs)

val equals (a b: payload) : boolean =
  compare(a, b) && compare(b, a)

val increment (v: integer, a: payload) : payload =
  make (GCounter_CRDT.increment(v, a.incs), a.decs)

val decrement (v: integer, a: payload) : payload =
  make (a.incs, GCounter_CRDT.increment(v, a.decs))
end
```

Listing E.2: PNCounter_CRDT

```
module VGCounter : CvRDT
  val size : integer

  assume size_pos : size >= 0

  type payload = integer

  type t = { array: map<integer, integer> } [vfx Vector[Int]]

  val create : t =
    { array: map.const(0) }

  val compute_payload (array: map<integer, integer>, replica: integer) : integer
    variant (size - replica)
    = if replica >= size || replica < 0 then 0
      else map.get(replica, array) + compute_payload(array, replica + 1)
```

```
val get_payload (a: t) : integer =
  compute_payload(a.array, 0)

val increment (replica: integer, a: t) : t =
  if replica >= 0 && replica < size then
    { array: map.set(replica, map.get(replica, a.array) + 1, a.array) }
  else
    a

val merge_array (m1 m2: map<integer, integer>) : map<integer, integer>
  ensures {
    forall(r: integer) {
      map.get(r, result) == max(map.get(r, m1), map.get(r, m2))
    }
  }

val merge (a b: t) : t =
  { array: merge_array(a.array, b.array) }

val compare (a b: t) : boolean =
  forall(r: integer) { map.get(r, a.array) <= map.get(r, b.array) }

val equals (a b: t) : boolean =
  compare(a, b) && compare(b, a)
end
```

Listing E.3: VGCounter

```
module OpCounter_CRDT : CmRDT
  type payload = integer

  type operation = Inc | Dec

  val create: payload = { payload : 0 }

  val execute (op: operation, counter: payload) : payload =
    match op with
    | Inc -> counter + 1
    | Dec -> counter - 1
    end

  val compare (a b: payload) : boolean =
    (a == b)

  val equals (a b: payload) : boolean =
    compare(a, b) && compare(b, a)
```

end

Listing E.4: OpCounter_CRDT

```
module NestedMap : CvRDT
  type k
  type v

  type payload = map<k, v>

  val bottom () : v

  val merge_values (v1 v2: v) : v
    axiom commutative
    axiom idempotent
    axiom associative

  val compare_values (v1 v2: v) : boolean
    axiom equivalence

  val create : payload =
    map.const(bottom())

  val add (key: k, value: v, a: payload) : payload =
    map.set(key, value, a)

  val remove (key: k, a: payload) : payload =
    map.set(key, bottom(), a)

  val merge_payload (m1 m2: payload) : payload =
    map.combine(m1, m2, merge_values)

  val merge (a b: payload) : payload =
    merge_payload(a, b)

  val compare (a b: payload) : boolean =
    forall(key: k) { compare_values(map.get(key, a), map.get(key, b)) }

  val equals (a b: payload) : boolean =
    a == b

  lemma compare_correct_aux (a b: payload)
    ensures { (compare(a, b) && compare(b, a)) <=> (a == b) }
  = true
end
```

Listing E.5: NestedMap

```
module GSet : CvRDT
  type elem
  type payload = set elem

  val create : payload =
    { payload : set.empty() }

  val add (v: elem, a: payload) : payload =
    set.add(v, a)

  val lookup (v: elem, a: payload) : boolean =
    set.contains(v, a)

  val merge (a b: payload) : payload =
    set.union(a, b)

  val compare (a b: payload) : boolean =
    set.subset(a, b)

  val equals (a b: payload) : boolean =
    compare(a, b) && compare(b, a)
end
```

Listing E.6: GSet

```
module TwoPSet : CvRDT
  type elem

  type payload =
    { added: set elem,
      removed: set elem }

  val create : payload =
    { added: set.empty(),
      removed: set.empty() }

  val add (v: elem, a: payload) : payload =
    { added: set.add(v, a.added),
      removed: a.removed }

  val remove (v: elem, a: payload) : payload =
    { added: a.added,
      removed: set.add(v, a.removed) }

  val lookup (v: elem, a: payload) : boolean =
    set.contains(v, a.added) && !set.contains(v, a.removed)
```

```
val merge (a b: payload) : payload =
  { added: set.union(a.added, b.added),
    removed: set.union(a.removed, b.removed) }

val compare (a b: payload) : boolean =
  set.subset(a.added, b.added) && set.subset(a.removed, b.removed)

val equals (a b: payload) : boolean =
  compare(a, b) && compare(b, a)
end
```

Listing E.7: TwoPSet

```
module OpTwoPSet : CmrDT
  type elem

  type operation = Add : elem | Rem : elem

  type payload =
    { added: set elem,
      removed: set elem }

  val create : payload =
    { added: set.empty(),
      removed: set.empty() }

  val execute (op: operation, a: payload) : payload =
    match op with
    | Add v -> { added: set.add(v, a.added), removed: a.removed }
    | Rem v -> { added: a.added, removed: set.add(v, a.removed) }
    end

  val lookup (v: elem, a: payload) : boolean =
    set.contains(v, a.added) && !set.contains(v, a.removed)

  val preRemove (v: elem, a: payload) [@vfx] : boolean =
    lookup(v, a)

  val enabledSrc (op: operation, a: payload) [@vfx] : boolean =
    match op with
    | Add v -> true
    | Rem v -> preRemove(v, a)
    end

  val forall_op (f: elem, a: payload) [@vfx] : boolean =
    set.diff(a.added, a.removed)
```

```

val filter (f: elem, a: payload) [@vfx] : boolean =
  set.diff(a.added, a.removed)

val compare (a b: payload) : boolean =
  set.subset(a.added, b.added) && set.subset(a.removed, b.removed)

val equals (a b: payload) : boolean =
  compare(a, b) && compare(b, a)
end

```

Listing E.8: OpTwoPSet

```

module UWMap : CvRDT
  type v

  type payload = map<integer, set v>

  type t = {
    added: map<integer, set (v, integer)>,
    removed: map<integer, set integer>
  }

  val create : t =
    { added: map.const(set.empty()),
      removed: map.const(set.empty()) }

  val get_ids (s: set (v, integer)) : set integer
  ensures {
    forall(i: integer) {
      set.contains(i, result) <-> exists(val_: v) { set.contains((val_, i), s) }
    }
  }

  val uwm_payload (added: map<integer, set (v, integer)>,
    removed: map<integer, set integer>) : map<integer, set v>
  ensures {
    forall(key: integer, val_: v) {
      set.contains(val_, map.get(key, result)) <->
      exists(id: integer) {
        set.contains((val_, id), map.get(key, added)) &&
        !set.contains(id, map.get(key, removed))
      }
    }
  }

  val get_payload (a: t) : map<integer, set v> =
    uwm_payload(a.added, a.removed)

```

```
val update (key: integer, value: v, id: integer, t: t) : t =
  { added: map.set(key, set.add((value, id), set.empty()), t.added),
    removed: map.set(key, set.union(map.get(key, t.removed),
      get_ids(map.get(key, t.added))), t.removed) }

val remove (key: integer, id: integer, t: t) : t =
  { removed: map.set(key, set.add(id, set.union(map.get(key, t.removed),
    get_ids(map.get(key, t.added)))), t.removed),
    added: map.set(key, set.empty(), t.added) }

val merge (a b: t) : t =
  { added: map.combine(a.added, b.added, set.union),
    removed: map.combine(a.removed, b.removed, set.union) }

val lookup (key: integer, t: t) : boolean =
  exists(val_: v, id: integer) {
    set.contains((val_, id), map.get(key, t.added)) &&
    !set.contains(id, map.get(key, t.removed))
  }

val compare (a b: t) : boolean =
  forall(key: integer) {
    set.subset(map.get(key, a.added), map.get(key, b.added)) &&
    set.subset(map.get(key, a.removed), map.get(key, b.removed))
  }

val equals (a b: t) : boolean =
  compare(a, b) && compare(b, a)

lemma compare_correct_aux (a b: t)
  ensures {
    (compare(a, b) && compare(b, a)) <->
    (a.added == b.added && a.removed == b.removed)
  }
= true
end
```

Listing E.9: UWMap

```
module RGA : CmRDT
  type lamport_clock =
    { replica: integer,
      counter: integer }
  @[vfx class : smaller_clock, increment_clock, sync,
    smaller_or_equal, bigger]
```

```

val smaller_clock (c1 c2: lamport_clock) : boolean =
  (c1.counter < c2.counter) ||
  (c1.counter == c2.counter && c1.replica < c2.replica)

type atom_type

val bottom_atom : atom_type

type elem =
  { atom: atom_type,
    timestamp: lamport_clock }

type payload =
  { clock: lamport_clock,
    added: set elem,
    removed: set elem,
    edges: map<elem, elem>,
    bottom_value: atom_type } [@vfx OpTwoPSet: vertices]

type source_op =
  AddRight: { l: elem, a: atom_type } | Remove: elem

type operation =
  Add: { l w: elem } | Rem: elem

val left_sentinel (bottom: atom_type) : elem =
  { atom: bottom, timestamp: { replica: -1, counter: -1 } }

val right_sentinel (bottom: atom_type) : elem =
  { atom: bottom, timestamp: { replica: -1, counter: 0 } }

val create : payload =
  { clock: { replica: 0, counter: 0 },
    added: set.empty(),
    removed: set.empty(),
    edges: map.set(left_sentinel(bottom_atom), right_sentinel(bottom_atom),
      map.const(right_sentinel(bottom_atom))),
    bottom_value: bottom_atom }

val lookup (v: elem, a: payload) : boolean =
  set.contains(v, a.added) && !set.contains(v, a.removed)

val successor (e: elem, a: payload) : elem =
  requires { lookup(e, a) } [@vfx]
  map.get(e, a.edges)

```

```
val preAddRight (l: elem, a: payload) [@vfx] : boolean =
  lookup(l, a) && l != right_sentinel(bottom_atom)

val preRemove (w: elem, a: payload) [@vfx] : boolean =
  lookup(w, a)

val enabledSrc (op: source_op, a: payload) [@vfx] : boolean =
  match op with
  | AddRight l atom -> preAddRight(l, a)
  | Remove w -> preRemove(w, a)
  end

val preAddRightDownstream (l: elem, a: payload) [@vfx] : boolean =
  set.contains(l, a.added)

val preRemoveDownstream (w: elem, a: payload) [@vfx] : boolean =
  set.contains(w, a.added)

val enabledDown (op: operation, a: payload) [@vfx] : boolean =
  match op with
  | Add l w -> preAddRightDownstream(l, a)
  | Rem w -> preRemoveDownstream(w, a)
  end

val before (a b: elem, t: payload) [@vfx] : boolean
=
  requires { lookup(a, t) && lookup(b, t) }
  if map.contains(a, t.edges) then
    map.get(a, t.edges) == b || before(map.get(a, t.edges), b, t)
  else
    false

val isNewTimestamp (t: lamport_clock, a: payload) [@vfx] : boolean =
  forall(vertex: elem) {
    !set.contains(vertex, a.added) || vertex.timestamp != t
  } &&
  forall(key: elem) {
    !map.contains(key, a.edges) ||
    (key.timestamp != t && (map.get(key, a.edges)).timestamp != t)
  } &&
  t != (left_sentinel(bottom_atom)).timestamp &&
  t != (right_sentinel(bottom_atom)).timestamp

val allVerticesBeforeRightSentinel (a: payload) [@vfx] : boolean =
  forall(v: elem) {
    !(lookup(v, a) && v != right_sentinel(bottom_atom)) ||
```

```

    before(v, right_sentinel(bottom_atom), a)
  }

val reachable (a: payload) [@vfx] : boolean =
  a.clock.replica >= 0 && a.clock.counter >= 0 &&
  forall(e: elem) {
    !(set.contains(e, a.added) && e != left_sentinel(bottom_atom) &&
      e != right_sentinel(bottom_atom)) ||
    (e.timestamp.replica >= 0 && e.timestamp.counter >= 0)
  } &&
  lookup(left_sentinel(bottom_atom), a) &&
  lookup(right_sentinel(bottom_atom), a) &&
  map.contains(left_sentinel(bottom_atom), a.edges) &&
  !exists(x: elem) { before(x, left_sentinel(bottom_atom), a) } &&
  exists(first: elem) {
    lookup(first, a) && map.get(left_sentinel(bottom_atom), a.edges) == first
  } &&
  exists(last: elem) {
    lookup(last, a) && map.contains(last, a.edges) &&
    map.get(last, a.edges) == right_sentinel(bottom_atom)
  } &&
  !map.contains(right_sentinel(bottom_atom), a.edges) &&
  before(left_sentinel(bottom_atom), right_sentinel(bottom_atom), a) &&
  !exists(x: elem, y: elem) {
    lookup(x, a) && lookup(y, a) && before(x, y, a) && before(y, x, a)
  } &&
  forall(x: elem) {
    !(lookup(x, a) && x != left_sentinel(bottom_atom) &&
      x != right_sentinel(bottom_atom)) ||
    (before(left_sentinel(bottom_atom), x, a) &&
      before(x, right_sentinel(bottom_atom), a))
  } &&
  forall(from: elem) {
    !map.contains(from, a.edges) ||
    (set.contains(from, a.added) && set.contains(map.get(from, a.edges), a.added))
  }

val find_insert_position (stamp: lamport_clock, e left right: elem,
                        a: payload, fuel: integer) : payload
  variant (fuel)
=
  if fuel <= 0 then
    { clock: a.clock,
      added: set.add(e, a.added),
      removed: a.removed,
      edges: map.set(e, right, map.set(left, e, a.edges)),

```

```
        bottom_value: a.bottom_value }
    else if smaller_clock(stamp, right.timestamp) then
        find_insert_position(stamp, e, right, successor(right, a), a, fuel - 1)
    else
        { clock: if smaller_clock(a.clock, e.timestamp) then e.timestamp else a.clock,
          added: set.add(e, a.added),
          removed: a.removed,
          edges: map.set(left, e, map.set(e, right, a.edges)),
          bottom_value: a.bottom_value }

val increment_clock (c: lamport_clock) [@vfx] : lamport_clock =
    { replica: c.replica, counter: c.counter + 1 }

val sync (c1 c2: lamport_clock) [@vfx] : lamport_clock =
    { replica: c1.replica,
      counter: (if c1.counter >= c2.counter then c1.counter
                else c2.counter) + 1 }

val smaller_or_equal (c1 c2: lamport_clock) [@vfx] : boolean =
    c1 == c2 || smaller_clock(c1, c2)

val bigger (c1 c2: lamport_clock) [@vfx] : boolean =
    (c1.counter > c2.counter) ||
    (c1.counter == c2.counter && c1.replica > c2.replica)

val make_elem (a: atom_type, c: lamport_clock) [@vfx] : elem =
    { atom: a, timestamp: c }

val addRight (l: elem, a: atom_type, t: payload) [@vfx] : operation =
    Add(l, make_elem(a, increment_clock(t.clock)))

val remove (w: elem, t: payload) [@vfx] : operation =
    Rem(w)

val prepare (op: source_op, t: payload) [@vfx] : operation =
    match op with
    | AddRight l a -> addRight(l, a, t)
    | Remove w -> remove(w, t)
    end

val addRightDownstream (l w: elem, a: payload) [@vfx] : payload =
    find_insert_position(w.timestamp, w, l, successor(l, a), a,
        set.cardinal(a.added) + 1)

val removeDownstream (w: elem, a: payload) [@vfx] : payload =
    { clock: a.clock, added: a.added, removed: set.add(w, a.removed),
```

```

    edges: a.edges, bottom_value: a.bottom_value }

val effect (op: operation, a: payload) [@vfx] : payload =
  match op with
  | Add l w -> addRightDownstream(l, w, a)
  | Rem w -> removeDownstream(w, a)
  end

val compatibleS (a b: payload) [@vfx] : boolean =
  a.clock.replica != b.clock.replica

val equals_extra (a b: payload) [@vfx] : boolean =
  a.edges == b.edges && a.bottom_value == b.bottom_value

val execute (op: operation, a: payload) : payload =
  match op with
  | Add left e -> find_insert_position(e.timestamp, e, left, successor(left, a),
    a, set.cardinal(a.added) + 1)
  | Rem e -> { clock: a.clock, added: a.added, removed: set.add(e, a.removed),
    edges: a.edges, bottom_value: a.bottom_value }
  end

val compare (a b: payload) : boolean =
  set.subset(a.added, b.added) && set.subset(a.removed, b.removed)

val equals (a b: payload) : boolean =
  compare(a, b) && compare(b, a)

lemma find_insert_position_facts (stamp: lamport_clock, e left right: elem,
  a: payload, fuel: integer)

  variant (fuel)
  ensures {
    (find_insert_position(stamp, e, left, right, a, fuel)).added ==
    set.add(e, a.added)
  }
  ensures {
    (find_insert_position(stamp, e, left, right, a, fuel)).removed == a.removed
  }
= if fuel <= 0 then true
  else if smaller_clock(stamp, right.timestamp) then
    find_insert_position_facts(stamp, e, right, successor(right, a), a, fuel - 1)
  else true

lemma fset_add_commute (x y: elem, s: set elem)
  ensures { set.add(x, set.add(y, s)) == set.add(y, set.add(x, s)) }
= true

```

```
lemma op_commutative (op1 op2: operation, a: payload)
  ensures { equals(execute(op2, execute(op1, a)), execute(op1, execute(op2, a))) }
= match op1, op2 with
  | Rem _, Rem _ -> true
  | Add _ _, Rem _ -> true
  | Rem _, Add _ _ -> true
  | Add _ _, Add _ _ -> true
end
end
```

Listing E.10: RGA

COLOPHON



This document was typeset with lua $\text{\LaTeX}$ and the **novathesis** template (v8.3.0), set in New PX (Palatino), with New PX Math for math.

J. M. Lourenço. *novathesis: A LaTeX Template for Academic Theses and Dissertations*. Version 8.3.0. NOVA University Lisbon. 2026. doi: [10.5281/zenodo.21938603](https://doi.org/10.5281/zenodo.21938603) This is *not* a PDF/A archival copy.

Build 12cc9022, September 22, 2026.

